Politechnika Wrocławska

SYSTEMY WBUDOWANE

Instrukcje laboratoryjne

Wersja bez sprzętu: implementacja i symulacja na PC
z kodem gotowym do przeniesienia na ESP8266 (MicroPython)

ZAWARTOŚĆ

0. Organizacja laboratorium
1. Python i MicroPython – środowisko, emulator, kod przenośny
2. Python – prosty konwerter systemów liczbowych
3. Sterowanie diodą LED – implementacja i testy
4. Przekaźnik czasowy sterowany przez WiFi (TELNET) – implementacja, symulacja i testy

Politechnika Wrocławska
Wersja dla studentów · rok akademicki 2026/2027

Politechnika Wrocławska

SYSTEMY WBUDOWANE – LABORATORIUM

Informacje ogólne – wersja bez sprzętu

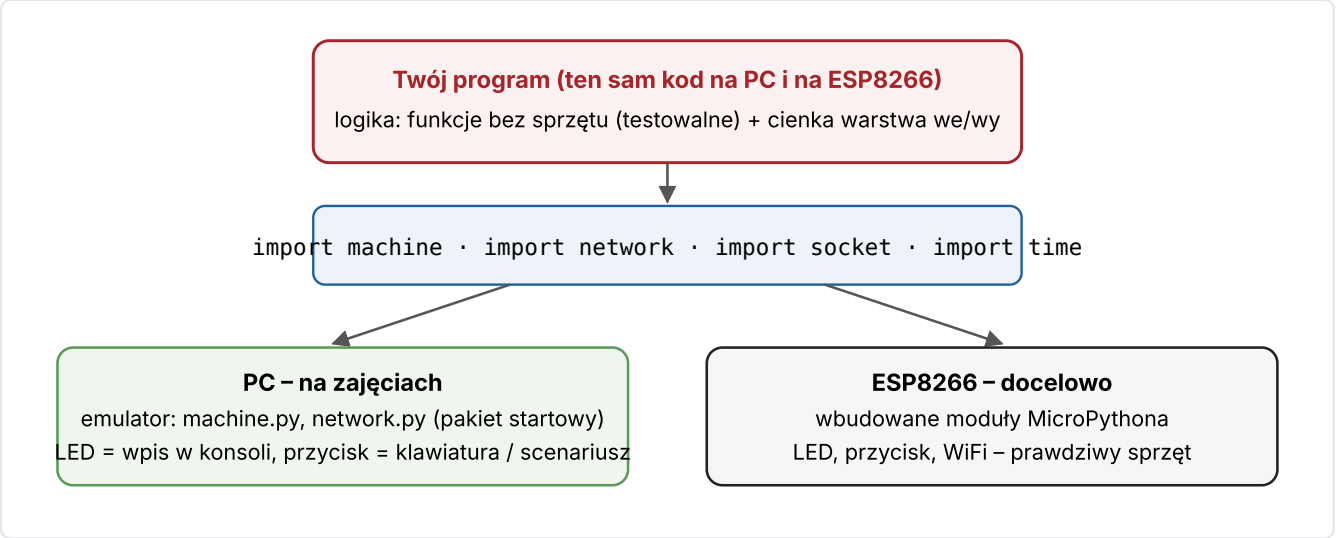
Organizacja laboratorium

Implementacja na PC z emulatorem ESP8266, kod gotowy do przeniesienia na urządzenie

1. Idea tej wersji laboratorium

W tej wersji **nie potrzebujesz modułu ESP8266**. Wszystkie programy piszesz i uruchamiasz na komputerze w Pythonie 3, ale tak, aby **ten sam kod** można było później wgrać na ESP8266 z MicroPythonem i uruchomić bez zmian (lub z minimalnymi zmianami opisanymi w każdej instrukcji).

Umożliwia to **emulator** z pakietu startowego – pliki `machine.py` i `network.py`, które udają moduły MicroPythona: zmiana stanu diody LED pojawia się w konsoli jako wpis z czasem, przycisk wciskasz klawiszem Enter (albo opisujesz scenariuszem wciśnięć w teście), a serwer sieciowy działa na Twoim komputerze pod adresem 127.0.0.1.



Dzięki temu uczysz się tego samego, co na sprzęcie: sterowania wyjściami, obsługi wejść, odmierzania czasu, komunikacji sieciowej – a dodatkowo **automatycznego testowania** oprogramowania wbudowanego, czego na samym urządzeniu zrobić się nie da.

Nr	Temat	Czas	Plik instrukcji
1	Python i MicroPython – środowisko, emulator, kod przenośny	1 jednostka	SW_SIM_01
2	Python – prosty konwerter systemów liczbowych	1 jednostka	SW_SIM_02
3	Sterowanie diodą LED (alfabet Morse'a) – implementacja i testy	1 jednostka	SW_SIM_03
4	Przekaźnik czasowy sterowany przez WiFi (TELNET) – implementacja, symulacja i testy	2 jednostki	SW_SIM_04

2. Pakiet startowy

Pobierz od prowadzącego archiwum `SW_pakiet_startowy.zip` i rozpakuj do własnego folderu roboczego. **Swoje programy zapisuj w tym samym folderze** – wtedy instrukcja `from machine import Pin` znajdzie emulator.

Plik	Do czego służy
`machine.py`	**emulator** modułu `machine`: `Pin`, `PWM`, `Timer` + funkcje czasu `time.sleep_ms`, `ticks_ms`, `ticks_diff` …
`network.py`	**emulator** modułu `network`: `WLAN` (tryby AP i STA)
`przyklad*.py`, `binled.py`, `automat.py`, `upper_server.py`, `automat_telnet.py`	przykładowe rozwiązania z instrukcji (działające)
`test_binled.py`, `test_automat.py`	przykłady testów automatycznych
`szablon_konwerter.py`, `szablon_morse.py`	szablony do zadań z laboratoriów 2 i 3
`test_morse.py`, `test_przekaznik.py`	**samosprawdzenie** zadań z laboratoriów 3 i 4
`telnet_srv.py`	punkt dostępowy + serwer TELNET z nieblokującym odczytem poleceń (laboratorium 4; wgrywany także na ESP8266)
`klient_telnet.py`	prosty klient TELNET (laboratorium 4)

WAŻNE

Pliki `machine.py` i `network.py` są **tylko dla PC**. Na prawdziwy ESP8266 ich **nie wgrywasz** – tam te moduły są wbudowane w MicroPythona.

3. Oprogramowanie

Program	Do czego służy	Uwagi
Python 3.10+	uruchamianie programów i testów	python.org; w Windows zaznacz „Add Python to PATH"
Thonny lub **VS Code**	edycja i uruchamianie kodu	w Thonny: *Narzędzia → Opcje → Interpreter → Lokalny Python 3*
terminal / wiersz poleceń	uruchamianie: `python plik.py`	w folderze z pakietem startowym
`matplotlib` (opcjonalnie)	wykresy przebiegów czasowych do sprawozdania	`pip install matplotlib`
PuTTY / `telnet` (opcjonalnie)	klient TELNET (lab. 4)	można zamiast nich użyć `klient_telnet.py`

Sprawdzenie instalacji – w folderze z pakietem wykonaj `python przyklad1_temperatura.py`. Jeśli polecenie `python` nie działa, spróbuj `py` (Windows) lub `python3` (Linux/macOS).

4. Emulator – najważniejsze informacje

4.1. Co widzisz w konsoli

```
[SIM] Przycisk GPIO14 sterowany klawiaturą: Enter = krótkie wciśnięcie (0,2 s), l+Enter =
długie (2,5 s), liczba+Enter = wciśnięcie na tyle ms.
[    0.000 s] LED (GPIO2) = 1   ○ zgaszona
[    1.204 s] Przycisk (GPIO14): WCIŚNIĘTY (0)
[    1.204 s] LED (GPIO2) = 0   ● ŚWIECI
[    1.405 s] Przycisk (GPIO14): puszczony (1)
```

Każda zmiana stanu pinu jest wypisywana z czasem od startu programu. GPIO2 (dioda wbudowana) jest **aktywna stanem niskim** – tak jak na prawdziwej płytce: wartość 0 = świeci.

4.2. Elementy emulowane

Element	Działanie w emulatorze
`Pin(n, Pin.OUT)`	`value()`, `on()`, `off()` – zmiana stanu wypisywana w konsoli i zapisywana w historii
`Pin(n, Pin.IN, Pin.PULL_UP)`	przycisk: **Enter** = wciśnięcie 0,2 s, **l + Enter** = 2,5 s, **liczba + Enter** = tyle ms; bez `PULL_UP` – ostrzeżenie (wejście „pływa")
`Pin.irq()`, `Timer`	obsługiwane, gdy program wywołuje funkcje `time.*` lub `Pin.value()` (np. w pętli z `sleep_ms`)
`PWM`	`duty()` 0–1023, `freq()`; jasność pokazywana jako pasek `[#####.....]`
`time.sleep_ms`, `ticks_ms`, `ticks_diff` …	dodawane do modułu `time` przy imporcie `machine` – **importuj `machine` przed użyciem tych funkcji**
`network.WLAN`	sprawdza konfigurację (np. długość hasła), adres serwera to 127.0.0.1
`socket`	prawdziwe gniazda – serwer działa na Twoim komputerze

4.3. Funkcje symulatora (tylko w testach i narzędziach – nie ma ich na ESP8266)

Funkcja	Działanie
`machine.sim_config(fast=True)`	**czas wirtualny**: `sleep_ms` nie czeka naprawdę – test 60-sekundowego przebiegu trwa ułamek sekundy
`machine.sim_config(log=False)`	wyłącza wypisywanie zmian stanu
`machine.sim_config(names={12: ("K", False)})`	nazwa pinu w logach (drugi element `True` = aktywny stanem niskim)
`machine.sim_button(14, [(1000, 200), (5000, 200)])`	scenariusz: wciśnięcia przycisku w 1. i 5. sekundzie, każde na 200 ms
`machine.sim_time()`	bieżący czas symulacji [ms]
`machine.sim_intervals(2)`	lista przedziałów `(start, koniec)` [ms], gdy wyjście było aktywne (np. dioda świeciła)
`machine.sim_ascii([14, 12], step_ms=100)`	przebieg czasowy narysowany w konsoli
`machine.sim_plot("przebieg.png", [14, 12])`	przebieg czasowy zapisany jako obraz – do sprawozdania

5. Zasady pisania kodu przenośnego (PC ↔ ESP8266)

REGUŁA NR 1 – ODDZIEL LOGIKĘ OD SPRZĘTU

Obliczenia, kodowanie, walidację danych i decyzje umieszczaj w **funkcjach, które nie dotykają sprzętu** (dostają argumenty, zwracają wynik). Obsługę pinów, `input()` i sieci trzymaj w kilku małych funkcjach. Program dziel na **moduł z logiką** (np. `morse.py`) i krótki `main.py`, który go uruchamia. Logikę testujesz na PC automatycznie, a na ESP8266 wgrywasz te same pliki.

Używaj	Unikaj (nie działa lub działa inaczej na ESP8266)
`str.format()` do formatowania tekstu	bibliotek spoza standardu: `numpy`, `pandas`, `requests`
`time.sleep_ms()`, `time.ticks_ms()`, `time.ticks_diff()`	`time.time()` do pomiaru krótkich odcinków, `datetime`
prostych struktur: `list`, `dict`, `str`, `int`, `float`	`threading`, `asyncio` (na ESP8266 brak wątków), `subprocess`, `os.system`
stałych z numerami pinów na początku pliku	`match … case`, adnotacji typów, `dataclasses`, `pathlib`
`sys.platform == "esp8266"` do rozróżnienia środowiska	głębokiej rekurencji i dużych list (ESP8266 ma ok. 30–40 kB RAM)
`try / except` przy konwersji danych od użytkownika	wywołań `machine.sim_…` poza plikami testów

6. Testy automatyczne

Test to zwykły program w Pythonie, który wywołuje Twoje funkcje i sprawdza wyniki instrukcją `assert` (lub funkcją wypisującą OK/BŁĄD). Testy uruchamiasz **tylko na PC**.

```python
# test_przyklad.py
import machine
machine.sim_config(fast=True, log=False)     # czas wirtualny, bez logów
import binled                                 # testowany moduł

assert binled.na_bity(11) == "1011"           # test logiki
start = machine.sim_time()
binled.nadaj("10")                            # test przebiegu czasowego
czasy = [round(b - a) for a, b in machine.sim_intervals(2) if a >= start]
assert czasy == [600, 150], czasy
print("Testy OK")
```

7. Przeniesienie na ESP8266 (gdy sprzęt będzie dostępny)

1. Wgraj firmware MicroPython na płytkę (Thonny: *Interpreter → MicroPython (ESP8266) → Zainstaluj lub zaktualizuj MicroPython*).
2. Prześlij **swoje moduły** i plik startowy jako `main.py` (Thonny: *Zapisz jako… → Urządzenie MicroPython*; lub `mpremote cp plik.py :plik.py`).
3. **Nie wgrywaj** `machine.py`, `network.py` ani plików `test_*.py`.
4. Sprawdź numery pinów (tabela w p. 9) i ustawienia zależne od platformy (np. port TELNET 23 zamiast 2323).
5. Uruchom (RST na płytce) i porównaj działanie z symulacją.

8. Zasady zaliczenia i wzór sprawozdania

1. Zadanie prezentujesz prowadzącemu **na zajęciach** (uruchomienie programu i testów + omówienie kodu).
2. Sprawozdanie w formie **PDF**, nazwa pliku: `SW_Lab<nr>_<Nazwisko>_<Imię>.pdf`; kod jako załącznik oraz pliki `.py`.
3. Kod musi być samodzielny – rozumiesz każdą linię. Źródła (dokumentacja, przykłady) podaj w sprawozdaniu.

Sekcja	Co powinno się w niej znaleźć
Strona tytułowa	przedmiot, nr i temat laboratorium, imię, nazwisko, nr indeksu, termin zajęć, data
1. Wstęp i opis problemu	3–6 zdań: co ma robić program, jakie są wymagania
2. Środowisko	system operacyjny, wersja Pythona (`python --version`), IDE, wersja emulatora, użyte moduły; docelowa platforma (ESP8266) i przypisanie pinów
3. Realizacja	struktura programu (moduły, funkcje, diagram stanów), podział na logikę i warstwę sprzętową, najważniejsze fragmenty kodu z komentarzem
4. Testy i prezentacja działania	zrzuty ekranu konsoli z podpisem, wynik testów automatycznych, przebiegi czasowe (`sim_ascii` / `sim_plot`)
5. Przeniesienie na urządzenie	co trzeba zmienić/wgrać, aby program działał na ESP8266; jakie mogą wystąpić różnice
6. Wnioski	co działa, co nie, jakie problemy napotkano i jak je rozwiązano
Załącznik	pełny kod programu i testów

9. Ściąga: wyprowadzenia płytki NodeMCU (ESP8266)

Kod piszesz dla docelowej płytki, więc używaj właściwych numerów GPIO.

Opis na płytce	GPIO (numer w `Pin()`)	Uwagi
D0	16	na NodeMCU – druga dioda LED (aktywna stanem 0); brak przerwań i PWM
D1 / D2	5 / 4	ogólnego przeznaczenia (też I^2C)
D3	0	przycisk FLASH; musi być 1 przy starcie – unikać
D4	2	**wbudowana dioda LED**, aktywna stanem 0
D5 / D6 / D7	14 / 12 / 13	ogólnego przeznaczenia – używane w lab. 4
D8	15	musi być 0 przy starcie – unikać
RX / TX	3 / 1	port szeregowy (REPL) – nie używać
A0	ADC0	wejście analogowe

Politechnika Wrocławska

SYSTEMY WBUDOWANE – LABORATORIUM

Laboratorium 1

Python i MicroPython – środowisko, emulator, kod przenośny

Konsola interaktywna, uruchamianie programów, pierwszy program przenośny z testami

Czas	1 jednostka laboratoryjna
Środowisko	komputer z Pythonem 3.10+, Thonny lub VS Code, pakiet startowy (emulator)
Oddajesz	prezentacja działania na zajęciach + sprawozdanie PDF + plik `funkcja.py`

1. Cel zajęć

Po zajęciach potrafisz:

- pracować w konsoli interaktywnej Pythona (**REPL**) i uruchamiać programy z pliku,
- korzystać z emulatora modułu `machine` i rozumiesz, co jest w nim symulowane,
- napisać prosty program **przenośny** (PC ↔ ESP8266) wraz z **testami automatycznymi**,
- opisać, jak taki program przenieść na moduł ESP8266.

2. Materiały do przeczytania przed zajęciami

- „Informacje ogólne – wersja bez sprzętu" (rozdziały 1–7).
- micropython.org → *Docs* → *Quick reference for the ESP8266* (przegląd możliwości modułu).
- MicroPython – różnice względem CPython (przejrzyj).

3. Podstawy

ESP8266 to mikrokontroler 32-bitowy z modułem WiFi, ok. 80 kB RAM (dla programu użytkownika ok. 30–40 kB) i 4 MB pamięci flash. **MicroPython** to implementacja Pythona 3 dla takich układów: składnia jest ta sama, różnią się biblioteki (`machine` , `network` , funkcje `time.sleep_ms` , `ticks_ms`) i dostępna pamięć.

Pojęcie	Na PC (ta wersja)	Na ESP8266
REPL – konsola `>>>`	`python` w terminalu albo okno *Powłoka* w Thonny	połączenie szeregowe 115200 bodów (Thonny, mpremote)
uruchomienie programu	`python plik.py` lub F5 w Thonny	plik `main.py` uruchamia się po starcie płytki
moduł `machine`	emulator `machine.py` z pakietu	wbudowany
przerwanie programu	**Ctrl+C**	**Ctrl+C**

4. Instrukcja krok po kroku

Krok 1 – Python i konsola REPL

Otwórz terminal w folderze z pakietem startowym i sprawdź wersję: `python --version`. Uruchom konsolę poleceniem `python` i wykonaj:

```
>>> import sys
>>> sys.version            # wersja Pythona – zapisz ją do sprawozdania
>>> sys.platform           # 'win32' / 'linux' / 'darwin'; na ESP8266 będzie 'esp8266'
>>> 2 ** 100               # liczby całkowite dowolnej wielkości
>>> 7 // 2, 7 % 2, 7 / 2   # dzielenie całkowite, reszta, dzielenie
>>> help(abs)
>>> exit()
```

Krok 2 – pierwszy kontakt z emulatorem

W konsoli (uruchomionej **w folderze z pakietem**) wpisz:

```
>>> from machine import Pin
>>> import time
>>> led = Pin(2, Pin.OUT)
>>> led.value(1)                    # dioda gaśnie (aktywna zerem)
>>> led.value(0)                    # dioda świeci
>>> for i in range(3):
...     led.value(1); time.sleep_ms(300); led.value(0); time.sleep_ms(300)
...
>>> t0 = time.ticks_ms(); time.sleep_ms(250); time.ticks_diff(time.ticks_ms(), t0)
```

Zaobserwuj wpisy `[ czas ] LED (GPIO2) = …` – tak emulator pokazuje stan diody.

Krok 3 – program w pliku

W Thonny ustaw interpreter *Lokalny Python 3*, otwórz `przyklad1_temperatura.py` i naciśnij **F5** (lub w terminalu: `python przyklad1_temperatura.py`). Program najpierw uruchamia testy, potem właściwe działanie.

> **WSKAZÓWKA**
>
> Jeśli pojawia się `ModuleNotFoundError: No module named 'machine'`, uruchamiasz program spoza folderu z pakietem. W Thonny zapisz swój plik w tym samym folderze co `machine.py`.

5. Przykład rozwiązania (inne zadanie)

PRZYKŁAD – PRZELICZANIE TEMPERATUR (PLIK PRZYKLAD1_TEMPERATURA.PY)

Treść: Zdefiniuj funkcje przeliczania stopni Celsjusza na Fahrenheita i odwrotnie, sprawdź je testami, wyświetl tabelę przeliczeń od −20 °C do 40 °C co 10 °C oraz przelicz wartość podaną przez użytkownika.

```
def c_na_f(c):
    """Zamienia stopnie Celsjusza na stopnie Fahrenheita: F = C * 9/5 + 32."""
    return c * 9 / 5 + 32

def f_na_c(f):
    """Zamienia stopnie Fahrenheita na stopnie Celsjusza: C = (F - 32) * 5/9."""
    return (f - 32) * 5 / 9

def testy():
    assert c_na_f(0) == 32
    assert c_na_f(100) == 212
    assert abs(f_na_c(100) - 37.78) < 0.01
    assert abs(f_na_c(c_na_f(21.5)) - 21.5) < 1e-9     # przeliczenie tam i z powrotem
    print("Testy OK")

def main():
    print("Tabela przeliczen C -> F")
    for c in range(-20, 41, 10):
        print("{:>5} C = {:>6.1f} F".format(c, c_na_f(c)))
    wartosc = float(input("Podaj temperature w F: "))
    print("{:.1f} F = {:.2f} C".format(wartosc, f_na_c(wartosc)))

testy()
main()
```

```
Testy OK
Tabela przeliczen C -> F
  -20 C =   -4.0 F
  -10 C =   14.0 F
    0 C =   32.0 F
   10 C =   50.0 F
   20 C =   68.0 F
   30 C =   86.0 F
   40 C =  104.0 F
Podaj temperature w F: 100
100.0 F = 37.78 C
```

Co warto zauważyć: obliczenia są w funkcjach, które nie wiedzą nic o klawiaturze i ekranie – dlatego da się je przetestować. Liczby zmiennoprzecinkowe porównujemy z tolerancją (`abs(a - b) < …`), nie przez `==` . Formatowanie przez `str.format()` działa także na ESP8266.

6. Zadanie do wykonania

ZADANIE

1. Napisz program `funkcja.py` realizujący **prostą funkcję matematyczną** – inną niż trójmian kwadratowy z wykładu i inną niż przykład. Wariant wybierz z tabeli: **numer wariantu = (numer na liście obecności − 1) mod 12 + 1**.
2. Program musi zawierać: (a) funkcję z parametrami zwracającą wynik (bez `input()` / `print()` w środku), (b) funkcję `testy()` z co najmniej **5 asercjami** (w tym przypadki brzegowe i błędne dane), (c) funkcję `main()` wyświetlającą wyniki dla min. 3 przykładów i przyjmującą dane od użytkownika z obsługą błędów.
3. Zmierz czas wykonania funkcji dla dużych danych przy użyciu `time.ticks_ms()` i `time.ticks_diff()` (pamiętaj o `import machine` przed `import time` na PC).
4. Kod ma być **przenośny** – zgodny z zasadami z „Informacji ogólnych", rozdz. 5.
5. Zaprezentuj działanie programu i testów prowadzącemu.

Wariant	Funkcja	Wskazówka
1	silnia *n!* (iteracyjnie i rekurencyjnie)	porównaj czas obu wersji
2	*n*-ty wyraz ciągu Fibonacciego	dla $n > 30$ wersja rekurencyjna jest bardzo wolna – zmierz!
3	NWD i NWW dwóch liczb	algorytm Euklidesa, NWW = a·b / NWD
4	test pierwszości + lista liczb pierwszych ≤ *n*	dzielniki sprawdzaj do √n
5	pole trójkąta ze wzoru Herona	sprawdź warunek istnienia trójkąta
6	odległość dwóch punktów i środek odcinka	`math.sqrt`
7	średnia, mediana, odchylenie standardowe listy	moduł `statistics` nie istnieje w MicroPythonie – napisz sam
8	pierwiastek kwadratowy metodą Newtona (Herona)	porównaj z `math.sqrt`
9	układ dwóch równań liniowych (wzory Cramera)	obsłuż wyznacznik = 0
10	szybkie potęgowanie a^n	zlicz liczbę mnożeń
11	suma cyfr liczby, podzielność przez 3 i 9	`% 10` i `// 10`
12	rata kredytu (raty równe)	$R = K \cdot q^n \cdot (q-1)/(q^n-1)$, $q = 1 + r/12$

PRZENIESIENIE NA ESP8266

Plik `funkcja.py` wgrany jako `main.py` uruchomi się na płytce bez zmian (konsola REPL w Thonny zastępuje terminal). Na urządzeniu zwróć uwagę na czas obliczeń – procesor 80 MHz jest kilkaset razy wolniejszy od PC – oraz na pamięć: duże listy (np. liczby pierwsze do 10^6) mogą zakończyć się `MemoryError`.

7. Sprawozdanie

- wstęp i opis problemu (funkcja, wzór matematyczny),
- środowisko: system, wersja Pythona, IDE, sposób uruchamiania,
- realizacja: struktura programu, opis testów (co sprawdza każdy test i dlaczego),
- prezentacja działania – zrzuty ekranu z komentarzem, wynik testów, wyniki pomiaru czasu,
- przeniesienie na ESP8266: co i jak należałoby wgrać; czego się spodziewasz (czas, pamięć),
- wnioski; załącznik: kod.

8. Lista kontrolna przed oddaniem

☐ Funkcja matematyczna nie używa `input()` ani `print()` – zwraca wynik.
☐ `testy()` zawiera min. 5 asercji i wypisuje „Testy OK".
☐ Program nie kończy się błędem (traceback) dla żadnych danych od użytkownika.
☐ Kod nie używa niczego z kolumny „Unikaj" (Informacje ogólne, rozdz. 5).
☐ Każdy zrzut ekranu ma podpis wyjaśniający, co pokazuje.

Politechnika Wrocławska

SYSTEMY WBUDOWANE – LABORATORIUM

Laboratorium 2

Python – prosty konwerter systemów liczbowych

Aplikacja konsolowa z testami: konwersja liczb dziesiętnych na system dwójkowy i szesnastkowy

Czas	1 jednostka laboratoryjna
Środowisko	komputer z Pythonem 3.10+, Thonny lub VS Code, pakiet startowy (`szablon_konwerter.py` , `przyklad2_jednostki.py`)
Oddajesz	prezentacja na zajęciach + sprawozdanie PDF + plik `konwerter.py`

1. Cel zajęć

Napisanie w Pythonie konsolowej aplikacji, która konwertuje liczby z systemu dziesiętnego na dwójkowy (BIN) oraz szesnastkowy (HEX), wraz z **testami automatycznymi**, w postaci **przenośnej** – gotowej do uruchomienia na module ESP8266 z MicroPythonem. Ćwiczymy: algorytmy konwersji, obsługę wejścia użytkownika, walidację danych, budowę prostego menu tekstowego, oddzielanie logiki od interfejsu użytkownika.

2. Teoria: pozycyjne systemy liczbowe

W systemie o podstawie p liczba zapisana cyframi $c_n \dots c_1 c_0$ ma wartość $c_n \cdot p^n + \dots + c_1 \cdot p^1 + c_0 \cdot p^0$. W systemie szesnastkowym cyfry 10–15 zapisujemy literami A–F.

Algorytm konwersji z dziesiętnego (dzielenie z resztą): dziel liczbę przez podstawę, zapisuj reszty, aż iloraz będzie równy 0. Wynik to reszty odczytane **od ostatniej do pierwszej**.

156 → system dwójkowy			156 → system szesnastkowy		
156 : 2 = 78	reszta 0		156 : 16 = 9	reszta 12 → C	
78 : 2 = 39	reszta 0		9 : 16 = 0	reszta 9	
39 : 2 = 19	reszta 1		Sprawdzenie: 9·16 + 12 = 156 ✓ Sprawdzenie BIN: 128+16+8+4 = 156 ✓ Przydatne: 1 cyfra HEX = 4 bity, np. 1001 1100 = 9 C		
19 : 2 = 9	reszta 1	odczyt ↑ 10011100_2			odczyt ↑ $9C_{16}$
9 : 2 = 4	reszta 1				
4 : 2 = 2	reszta 0				
2 : 2 = 1	reszta 0				
1 : 2 = 0	reszta 1				

Przydatne elementy Pythona

Element	Przykład	Wynik / uwagi
dzielenie całkowite i reszta	`156 // 16`, `156 % 16`	`9`, `12`
cyfry do HEX	`"0123456789ABCDEF"[12]`	`'C'`
wczytanie tekstu	`s = input("Podaj liczbę: ")`	zawsze zwraca **tekst** (`str`)
zamiana na liczbę	`int(s)`	rzuca `ValueError`, gdy tekst nie jest liczbą
obsługa błędu	`try: ... except ValueError: ...`	zob. przykład niżej
wbudowane konwersje (do **sprawdzenia** wyniku)	`bin(156)`, `hex(156)`	`'0b10011100'`, `'0x9c'`
pętla menu	`while True: ... break`	`break` kończy pętlę

WYMAGANIE DOTYCZĄCE ALGORYTMU

Konwersję zaimplementuj **samodzielnie** (algorytm dzielenia z resztą). Funkcji `bin()`, `hex()` i `format()` możesz użyć tylko do automatycznego sprawdzenia poprawności swojego wyniku (np. w funkcji testującej).

3. Przykład rozwiązania (inne zadanie)

PRZYKŁAD – KONWERTER JEDNOSTEK DŁUGOŚCI Z MENU (PLIK PRZYKLAD2_JEDNOSTKI.PY)

Treść: Program prosi o długość w metrach (zakres 0–10000), następnie o wybór jednostki docelowej (centymetry / cale), wyświetla wynik i pyta o dalsze działanie. Błędne dane są sygnalizowane komunikatem.

```
# jednostki.py – przykład struktury aplikacji konsolowej z menu i walidacją
MIN_W, MAX_W = 0, 10000

def wczytaj_liczbe(komunikat, minimum, maksimum):
    """Pyta tak długo, aż użytkownik poda liczbę z zakresu <minimum, maksimum>."""
    while True:
        tekst = input(komunikat).strip()
        try:
            wartosc = float(tekst)
        except ValueError:
            print("  Blad: '{}' nie jest liczba.".format(tekst))
            continue
        if minimum <= wartosc <= maksimum:
            return wartosc
        print("  Blad: dozwolony zakres to {}..{}.".format(minimum, maksimum))

def wczytaj_wybor(komunikat, dozwolone):
    """Zwraca jedną z dozwolonych opcji (wielkość liter bez znaczenia)."""
    while True:
        w = input(komunikat).strip().upper()
        if w in dozwolone:
            return w
        print("  Blad: wybierz jedna z opcji: {}".format(", ".join(dozwolone)))

def na_cm(m):
    return m * 100

def na_cale(m):
    return m / 0.0254

def main():
    print("=== Konwerter jednostek dlugosci ===")
    metry = wczytaj_liczbe("Podaj dlugosc w metrach ({}-{}): ".format(MIN_W, MAX_W),
MIN_W, MAX_W)
    while True:
        cel = wczytaj_wybor("Na jaka jednostke? [C] cm, [I] cale: ", ["C", "I"])
        if cel == "C":
            print("  {} m = {:.2f} cm".format(metry, na_cm(metry)))
        else:
            print("  {} m = {:.2f} cali".format(metry, na_cale(metry)))
        dalej = wczytaj_wybor("[D] druga jednostka, [N] nowa liczba, [K] koniec: ", ["D",
"N", "K"])
        if dalej == "K":
            print("Do widzenia!")
            break
        if dalej == "N":
            metry = wczytaj_liczbe("Podaj dlugosc w metrach: ", MIN_W, MAX_W)

main()
```

Przykładowa sesja:

```
=== Konwerter jednostek dlugosci ===
Podaj dlugosc w metrach (0-10000): abc
  Blad: 'abc' nie jest liczba.
Podaj dlugosc w metrach (0-10000): 2.5
Na jaka jednostke? [C] cm, [I] cale: x
```

```
  Blad: wybierz jedna z opcji: C, I
Na jaka jednostke? [C] cm, [I] cale: c
  2.5 m = 250.00 cm
[D] druga jednostka, [N] nowa liczba, [K] koniec: d
Na jaka jednostke? [C] cm, [I] cale: i
  2.5 m = 98.43 cali
[D] druga jednostka, [N] nowa liczba, [K] koniec: k
Do widzenia!
```

Zwróć uwagę na podział programu na **małe funkcje**: osobno wczytywanie z walidacją, osobno obliczenia, osobno logika menu. Ten sam schemat zastosuj w swoim konwerterze.

4. Zadanie do wykonania

ZADANIE

Napisz w Pythonie konsolową aplikację `konwerter.py` (punkt wyjścia: `szablon_konwerter.py`) przeliczającą liczby całkowite z systemu dziesiętnego na dwójkowy oraz szesnastkowy. Kod ma być **przenośny** (zasady z „Informacji ogólnych", rozdz. 5) i zawierać testy automatyczne. Działanie zaprezentuj na kilku przykładach ilustrujących wszystkie funkcjonalności.

Wymagania obowiązkowe

1. Po uruchomieniu użytkownik widzi prośbę o podanie liczby dziesiętnej z zakresu **0–65535** (liczba 16-bitowa bez znaku) i może ją wprowadzić.
2. Po podaniu liczby użytkownik widzi prośbę o wskazanie systemu docelowego (dwójkowy lub szesnastkowy) i może dokonać wyboru.
3. Po wyborze systemu użytkownik widzi **poprawny wynik konwersji**, obliczony własnym algorytmem.

Wymagania dodatkowe (rozszerzenie)

4. Pytanie o kolejne akcje: przeliczenie tej samej liczby na drugi system, podanie nowej liczby albo zakończenie programu.
5. Obsługa błędów: komunikaty o niepoprawnej wartości (tekst zamiast liczby, liczba spoza zakresu, liczba ujemna, zły wybór w menu).
6. Wynik dwójkowy wyświetlany w grupach po 4 bity (np. `1001 1100`).

Wymagania dotyczące struktury i testów

7. Logika (konwersja, walidacja tekstu wpisanego przez użytkownika, grupowanie bitów) w funkcjach **bez** `input()` i `print()`; interakcja z użytkownikiem tylko w `main()` i funkcjach wczytujących.
8. Funkcja `test()` sprawdza konwersję (porównanie z `bin()` / `hex()` dla min. 10 wartości, w tym brzegowych) oraz walidację (min. 5 przypadków błędnych danych).

Szkielet programu (plik szablon_konwerter.py)

```python
# konwerter.py — szkielet
CYFRY = "0123456789ABCDEF"
MIN_L, MAX_L = 0, 65535

def dec_na_system(liczba, podstawa):
```

```
    """Zwraca tekst z zapisem liczby w systemie o podanej podstawie (2 lub 16)."""
    if liczba == 0:
        return "0"
    wynik = ""
    while liczba > 0:
        # TODO: wyznacz resztę z dzielenia przez podstawę,
        #       dopisz odpowiednią cyfrę z CYFRY na POCZĄTEK wyniku,
        #       podziel liczbę całkowicie przez podstawę
        break          # TODO: usuń tę linię po uzupełnieniu pętli
    return wynik

def test():
    """Automatyczne sprawdzenie algorytmu funkcjami wbudowanymi."""
    for n in (0, 1, 2, 10, 15, 16, 156, 255, 256, 65535):
        assert dec_na_system(n, 2) == bin(n)[2:], n
        assert dec_na_system(n, 16) == hex(n)[2:].upper(), n
    print("Testy OK")

# TODO: funkcje wczytujące dane z walidacją (por. przykład z rozdziału 3)
# TODO: funkcja main() z pętlą menu

test()
# main()
```

WSKAZÓWKI

- Walidację łatwo przetestować, jeśli funkcja przyjmuje **tekst** i zwraca wynik albo komunikat błędu, np. `sprawdz_liczbe("abc") -> (None, "to nie jest liczba")`, `sprawdz_liczbe("156") -> (156, None)`. Funkcja wczytująca tylko wywołuje ją w pętli z `input()`.
- Polskie znaki w komunikatach mogą wyświetlać się niepoprawnie w niektórych terminalach – jeśli tak się dzieje, pisz bez nich.
- Dobrze dobrane przykłady testowe: 0, 1, 15, 16, 255, 256, 65535 (wartości graniczne) oraz −1, 65536, `abc`, `12.5`, pusty tekst (błędy).
- Wynik w grupach po 4 bity: uzupełnij tekst zerami z lewej do wielokrotności 4, a potem wstawiaj spację co 4 znaki.

PRZENIESIENIE NA ESP8266

Program nie używa sprzętu, więc wystarczy wgrać `konwerter.py` (jako `main.py` lub uruchomić z Thonny przy interpreterze *MicroPython (ESP8266)*) – `input()` i `print()` działają w konsoli REPL tak samo jak na PC. Funkcje `bin()` i `hex()` są dostępne w MicroPythonie, więc `test()` można wykonać także na płytce.

5. Sprawozdanie

- zwięzły opis problemu,
- środowisko: system, wersja Pythona, IDE, wszystkie wykorzystane narzędzia; docelowa platforma (ESP8266/MicroPython) i sposób przeniesienia,
- opis działania aplikacji wraz ze zrzutami ekranu (przykłady reprezentatywne dla zaimplementowanych funkcjonalności); ewentualnie przykłady błędów oraz ich analiza,
- opis testów i ich wynik,
- wnioski,
- załącznik: kod programu.

6. Lista kontrolna przed oddaniem

☐ Konwersja jest wykonywana własnym algorytmem, a `test()` wypisuje „Testy OK”.
☐ Logika jest oddzielona od `input()` / `print()`; kod spełnia zasady przenośności.
☐ Program nie kończy się błędem (traceback) dla żadnych danych wpisanych przez użytkownika.
☐ Zrzuty ekranu pokazują konwersję na BIN, na HEX, zmianę systemu, nową liczbę, zakończenie i obsługę błędów.

Politechnika Wrocławska

SYSTEMY WBUDOWANE – LABORATORIUM

Laboratorium 3

Sterowanie diodą LED – implementacja i testy

Wyjścia cyfrowe i PWM, funkcje czasowe, nadawanie słów alfabetem Morse'a – na emulatorze

Czas	1 jednostka laboratoryjna
Środowisko	Python 3.10+, pakiet startowy: `machine.py`, `szablon_morse.py`, `test_morse.py`, `binled.py`
Platforma docelowa	ESP8266 – wbudowana dioda LED na GPIO2
Oddajesz	prezentacja na zajęciach + sprawozdanie PDF + pliki `morse.py`, `main.py`

1. Zagadnienia

- obsługa wyjść cyfrowych oraz „analogowych" (PWM) modułu ESP8266 – na emulatorze,
- identyfikacja oznaczeń portów modułu ESP8266 na podstawie dokumentacji,
- sterowanie diodą wbudowaną: włączenie, wyłączenie, funkcje czasowe (TON, TOFF, DELAY) – realizacja w (Micro)Pythonie,
- aplikacja sterująca diodą: kodowanie słów alfabetem Morse'a,
- **weryfikacja przebiegów czasowych testem automatycznym** (czas wirtualny emulatora).

Przydatne linki: MicroPython on ESP8266 Workshop – Basics · micropython.org – Quick reference ESP8266

2. Teoria

2.1. Oznaczenia portów i dioda wbudowana

Napisy na płytce (D0...D8) **nie są** numerami używanymi w `Pin()`. W kodzie podajemy numer **GPIO** (tabela w „Informacjach ogólnych", rozdz. 9). Emulator stosuje te same numery i tę samą logikę. Dioda wbudowana w moduł ESP-12 jest podłączona do **GPIO2 (D4)** i jest **aktywna stanem niskim**: `value(0)` – świeci, `value(1)` – zgaszona. Na płytce NodeMCU jest druga dioda na GPIO16 (D0).

```
from machine import Pin
led = Pin(2, Pin.OUT)

def led_on():  led.value(0)   # logika odwrócona!
def led_off(): led.value(1)
```

2.2. „Wyjście analogowe" – PWM

ESP8266 nie ma przetwornika C/A. Jasność diody reguluje się **modulacją szerokości impulsu (PWM)**: pin bardzo szybko przełącza się między 0 i 1, a jasność zależy od współczynnika wypełnienia. W MicroPythonie dla ESP8266: `duty` 0–1023, częstotliwość 1 Hz – 1 kHz.

```
from machine import Pin, PWM
import time
pwm = PWM(Pin(2), freq=1000)
for d in range(0, 1024, 32):      # "oddychanie" diody
    pwm.duty(1023 - d)            # dioda aktywna zerem: 1023 = zgaszona
    time.sleep_ms(20)
pwm.deinit()                      # zwolnienie pinu
```

W emulatorze każda zmiana wypełnienia jest pokazywana jako pasek jasności, np. `jasność [#####.....] 50%`.

2.3. Funkcje czasowe

Nazwy TON, TOFF (TOF), TP pochodzą ze sterowników PLC (norma IEC 61131-3). W MicroPythonie realizujemy je samodzielnie przy użyciu modułu `time`.

Funkcja	Działanie	Realizacja w MicroPythonie
DELAY	wstrzymanie programu na zadany czas	`time.sleep(s)`, `time.sleep_ms(ms)` – **blokujące**
TON (opóźnione załączenie)	wyjście = 1, gdy wejście jest 1 nieprzerwanie przez czas PT	pomiar czasu: `ticks_ms()` + `ticks_diff()`
TOFF (opóźnione wyłączenie)	wyjście pozostaje 1 jeszcze przez PT po zaniku wejścia	j.w.
TP (impuls)	po zboczu narastającym wejścia wyjście = 1 dokładnie przez PT	j.w.
pomiar czasu	ile ms upłynęło od zdarzenia	`t0 = time.ticks_ms()` … `time.ticks_diff(time.ticks_ms(), t0)`
zdarzenie cykliczne	wywołanie funkcji co *n* ms w tle	`machine.Timer(-1).init(period=n, callback=f)` (w emulatorze wywoływana, gdy program korzysta z `time.*`)

BLOKUJĄCE VS NIEBLOKUJĄCE

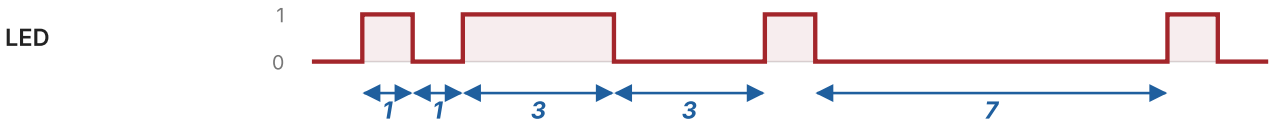
`sleep_ms()` jest proste, ale w tym czasie program nie robi nic innego (np. nie sprawdza przycisku). Wersja nieblokująca sprawdza w pętli, czy minął zadany czas. Licznik `ticks_ms()` „przekręca się", dlatego różnicę zawsze licz przez `ticks_diff()`, nie zwykłym odejmowaniem.

```
import time
class TON:
    """Timer opóźnionego załączenia (jak w PLC)."""
    def __init__(self, pt_ms):
        self.pt = pt_ms
        self.start = None
    def update(self, wejscie):
        if not wejscie:                 # brak sygnału -> reset timera
            self.start = None
            return False
        if self.start is None:          # zbocze narastające -> start pomiaru
            self.start = time.ticks_ms()
        return time.ticks_diff(time.ticks_ms(), self.start) >= self.pt
```

2.4. Alfabet Morse'a – zasady czasowe

Wszystkie czasy są wielokrotnościami **jednostki T** (np. T = 200 ms):

Element	Czas	Element	Czas
kropka `.` – dioda świeci	1 T	przerwa między elementami jednej litery	1 T
kreska `-` – dioda świeci	3 T	przerwa między literami	3 T
		przerwa między słowami	7 T



„A" (·–), przerwa międzyliterowa (3), „E" (·), przerwa między słowami (7) – wartości w jednostkach T.

A `.-`	B `-...`	C `-.-.`	D `-..`	E `.`	F `..-.`	G `--.`	H `....`
I `..`	J `.---`	K `-.-`	L `.-..`	M `--`	N `-.`	O `---`	P `.--.`
Q `--.-`	R `.-.`	S `...`	T `-`	U `..-`	V `...-`	W `.--`	X `-..-`
Y `-.--`	Z `--..`	Ą `.-.-`	Ć `-.-..`	Ę `..-..`	Ł `.-..-`	Ń `--.--`	Ó `---.`
Ś `...-...`	Ź `--..-.`	Ż `--..-`					

WSKAZÓWKA – SŁOWNIK

Najwygodniej zapisać alfabet w słowniku: `MORSE = {"A": ".-", "B": "-...", ...}`. Kod litery to `MORSE[litera]`, a sprawdzenie, czy znak jest dozwoloną literą: `litera in MORSE`. Kod całego słowa: `" ".join(MORSE[z] for z in slowo.upper())`.

2.5. Jak zobaczyć i sprawdzić przebieg czasowy na PC

Emulator zapisuje historię zmian stanu pinów. Na jej podstawie możesz narysować przebieg albo sprawdzić czasy w teście:

```
import machine
machine.sim_config(fast=True, log=False)   # czas wirtualny, bez logów
import binled
binled.nadaj("10")
machine.sim_ascii([2], step_ms=50)     # rysunek w konsoli
print([(round(a), round(b)) for a, b in machine.sim_intervals(2)])   # kiedy dioda świeciła [ms]
```

```
Przebieg czasowy: 1 znak = 50 ms, od 0.0 s do 1.6 s
      LED |#############_______####________
[(0, 600), (1000, 1150)]
```

3. Ćwiczenia wstępne (na rozgrzewkę, ok. 30 min)

Wykonaj jako krótkie programy w folderze z pakietem; wyniki (logi emulatora) pokaż prowadzącemu i opisz w sprawozdaniu.

1. **Miganie:** dioda GPIO2 miga 10 razy z częstotliwością 1 Hz (0,5 s świeci, 0,5 s nie świeci), potem gaśnie. Sprawdź w logu, czy odstępy wynoszą 500 ms.
2. **DELAY / TON / TOFF:** po uruchomieniu dioda zapala się po 3 s, świeci 2 s i gaśnie. Zrealizuj to dwukrotnie: raz `sleep()`, raz nieblokująco z `ticks_ms()` (np. klasą `TON`).
3. **PWM:** program pyta o poziom jasności 0–4 i ustawia wypełnienie 0 %, 25 %, 50 %, 75 %, 100 % (pamiętaj o logice odwróconej diody); `q` kończy program.
4. **Nieblokujące miganie:** dioda miga co 300 ms, a jednocześnie program co sekundę wypisuje licznik sekund (bez `sleep` dłuższego niż kilka ms w pętli głównej).
5. **Test:** dla ćwiczenia 1 napisz test w trybie `fast=True`, który sprawdzi przez `sim_intervals(2)`, że było 10 błysków po 500 ms.

4. Przykład rozwiązania (inne zadanie)

PRZYKŁAD – LICZBA W KODZIE DWÓJKOWYM NADAWANA DIODĄ (PLIKI BINLED.PY, PRZYKLAD3_MAIN.PY, TEST_BINLED.PY)

Treść: Program prosi o liczbę 0–15, wypisuje jej zapis 4-bitowy i „nadaje" go diodą: bit 1 = długi błysk (600 ms), bit 0 = krótki (150 ms), przerwa między bitami 400 ms. `q` kończy program. Logika jest w module `binled.py`, a uruchamia ją krótki `przyklad3_main.py` (na ESP8266: `main.py`).

```
# binled.py – moduł z logiką
from machine import Pin
import time

DLUGI, KROTKI, PRZERWA = 600, 150, 400
led = Pin(2, Pin.OUT, value=1)       # value=1 -> dioda (aktywna zerem) zgaszona na starcie

def blysk(ms):
    led.value(0)                     # świeci
    time.sleep_ms(ms)
    led.value(1)                     # gaśnie
    time.sleep_ms(PRZERWA)

def na_bity(n):
    """Zwraca 4-znakowy zapis dwójkowy liczby 0..15 (logika – bez sprzętu, łatwa do
testowania)."""
    bity = ""
    for _ in range(4):
        bity = str(n % 2) + bity
        n //= 2
    return bity

def nadaj(bity):
    for b in bity:
        blysk(DLUGI if b == "1" else KROTKI)

def main():
    while True:
        tekst = input("Liczba 0-15 (q - koniec): ").strip()
        if tekst.lower() == "q":
            break
        if not tekst.isdigit() or not 0 <= int(tekst) <= 15:
            print("  To nie jest liczba z zakresu 0-15.")
            continue
        bity = na_bity(int(tekst))
        print("  {} = {} (nadaje...)".format(tekst, bity))
        nadaj(bity)
    print("Koniec programu.")
```

```
# przyklad3_main.py (na ESP8266: main.py)
import binled
binled.main()
```

Działanie na emulatorze (`python przyklad3_main.py`; czasy liczone od startu programu):

```
[    0.000 s] LED (GPIO2) = 1   ○ zgaszona
Liczba 0-15 (q - koniec): 5
  5 = 0101 (nadaje...)
[    0.001 s] LED (GPIO2) = 0   ● ŚWIECI
[    0.151 s] LED (GPIO2) = 1   ○ zgaszona
[    0.551 s] LED (GPIO2) = 0   ● ŚWIECI
[    1.151 s] LED (GPIO2) = 1   ○ zgaszona
[    1.552 s] LED (GPIO2) = 0   ● ŚWIECI
[    1.702 s] LED (GPIO2) = 1   ○ zgaszona
```

```
[    2.102 s] LED (GPIO2) = 0   ● ŚWIECI
[    2.702 s] LED (GPIO2) = 1   ○ zgaszona
Liczba 0-15 (q - koniec): q
Koniec programu.
```

Test automatyczny (`python test_binled.py`) – trwa ułamek sekundy dzięki czasowi wirtualnemu:

```
import machine
machine.sim_config(fast=True, log=False)
import binled

assert binled.na_bity(0) == "0000"
assert binled.na_bity(11) == "1011"
assert binled.na_bity(15) == "1111"

start = machine.sim_time()
binled.nadaj("1011")
blyski = [(round(a - start), round(b - a)) for a, b in machine.sim_intervals(2) if a >=
start]
print("Błyski (start [ms], czas [ms]):", blyski)
assert [d for _, d in blyski] == [600, 150, 600, 600], "zle czasy blyskow"
assert blyski[1][0] - (blyski[0][0] + blyski[0][1]) == 400, "zla przerwa"
print("Testy OK")
```

```
Błyski (start [ms], czas [ms]): [(0, 600), (1000, 150), (1550, 600), (2550, 600)]
Testy OK
```

5. Zadanie do wykonania

ZADANIE – NADAJNIK ALFABETU MORSE'A

Napisz konsolową aplikację (przenośną: PC z emulatorem ↔ ESP8266), która:

1. prosi użytkownika o wpisanie z klawiatury **słowa**, rozumianego jako ciąg liter (bez cyfr, spacji i znaków specjalnych),
2. sprawdza, czy podany ciąg znaków stanowi słowo według powyższej definicji – jeśli nie, informuje, co jest niedozwolone, i prosi ponownie,
3. koduje wpisane znaki alfabetem Morse'a (**zalecane użycie słownika**), wypisuje ciąg kropek i kresek na konsoli (litery rozdzielone spacją) oraz **nadaje go wbudowaną diodą LED** (GPIO2) zgodnie z zasadami czasowymi z p. 2.4,
4. prosi o podanie kolejnego słowa,
5. na każdym etapie umożliwia zakończenie działania aplikacji (np. `q` / `koniec`).

Wymagana struktura (interfejs sprawdzany testem): moduł `morse.py` (punkt wyjścia: `szablon_morse.py`) zawiera stałą `T` [ms], słownik `MORSE` oraz funkcje:

- `sprawdz(tekst)` → `None`, gdy tekst jest poprawnym słowem, w przeciwnym razie tekst komunikatu błędu,
- `zakoduj(slowo)` → kod Morse'a, litery rozdzielone pojedynczą spacją, np. `"SOS"` → `"... --- ..."`,
- `nadaj(kod)` → nadaje kod diodą; po zakończeniu dioda jest zgaszona.

Pętla z obsługą użytkownika jest w `main.py`. **Warunek zaliczenia:** `python test_morse.py` kończy się komunikatem „wszystkie testy zaliczone".

Rozszerzenia (dla chętnych): obsługa polskich liter; zmiana tempa (`tempo 150`); nadawanie nieblokujące (generator stanów + pętla z `ticks_ms`), dzięki któremu program może w trakcie nadawania reagować na przycisk.

Szkielet (plik szablon_morse.py → zapisz jako morse.py)

```
from machine import Pin
import time

T = 200                                  # jednostka czasu [ms]
led = Pin(2, Pin.OUT, value=1)           # dioda wbudowana, aktywna stanem 0

MORSE = {
    "A": ".-", "B": "-...", "C": "-.-.",  # TODO: uzupełnij pozostałe litery
}

def sprawdz(tekst):
    """Zwraca None, gdy tekst jest poprawnym słowem, w przeciwnym razie komunikat błędu."""
    # TODO

def zakoduj(slowo):
    """Zwraca kod Morse'a słowa, litery rozdzielone spacją, np. 'SOS' -> '... --- ...'."""
    # TODO

def nadaj(kod):
    """Nadaje kod diodą: '.' = 1T, '-' = 3T, przerwa w literze 1T, między literami 3T."""
    # TODO
```

Przykładowy wynik samosprawdzenia (`python test_morse.py`) dla poprawnego rozwiązania:

```
1. Sprawdzanie słowa
  [OK]   'Ala' jest słowem
  [OK]   'Ala1' odrzucone (cyfra)
  ...
3. Nadawanie diodą (kod '.- .' = litery A i E)
     błyski [ms]: [200, 600, 200]  przerwy [ms]: [200, 600]
  [OK]   kropka = 1T, kreska = 3T
  [OK]   przerwa wewnątrz litery = 1T
  [OK]   przerwa między literami = 3T
  [OK]   dioda zgaszona po nadaniu
WYNIK: wszystkie testy zaliczone
```

WSKAZÓWKI

- Litery małe i wielkie traktuj jednakowo – `slowo.upper()` . Pusty tekst też nie jest słowem.
- Przerwa między literami to 3 T, ale po każdym elemencie już odczekałeś 1 T – wystarczy więc dodać 2 T.
- Najpierw doprowadź do zaliczenia części 1 i 2 testu (logika), dopiero potem nadawanie.
- Na ekranie „SOS” powinno dać `... --- ...` ; w logu emulatora sprawdź rytm: 3 krótkie, 3 długie, 3 krótkie.
- Do sprawozdania: `machine.sim_ascii([2], step_ms=100)` lub `machine.sim_plot("sos.png", [2])` po nadaniu słowa.

PRZENIESIENIE NA ESP8266

Wgraj `morse.py` i `main.py` (bez `machine.py` i testów). Dioda na GPIO2 zachowa się tak jak w logu emulatora. Różnica: na płytce `input()` czyta z konsoli REPL (Thonny), a czasy mogą odbiegać o pojedyncze milisekundy.

6. Sprawozdanie

Wg wzoru z „Informacji ogólnych”. Dodatkowo: tabela GPIO platformy docelowej, opis realizacji funkcji czasowych (blokująca vs nieblokująca), wyniki ćwiczeń wstępnych, zrzuty ekranu: poprawne słowo (kod + log diody), błędne słowo, zakończenie programu, wynik `test_morse.py` , przebieg czasowy jednego słowa (`sim_ascii` / `sim_plot`).

7. Lista kontrolna przed oddaniem

- ☐ `test_morse.py` – wszystkie testy zaliczone.
- ☐ Słowa z cyframi, spacjami i znakami specjalnymi są odrzucane z czytelnym komunikatem.
- ☐ Aplikację można zakończyć zarówno przy wpisywaniu słowa, jak i po nadaniu.
- ☐ Kodowanie jest w module `morse.py` , a `main.py` tylko go używa.

Politechnika Wrocławska

SYSTEMY WBUDOWANE – LABORATORIUM

Laboratorium 4

Przekaźnik czasowy sterowany przez WiFi (TELNET) – implementacja, symulacja i testy

Punkt dostępowy WiFi, serwer TELNET, przekaźnik czasowy z obsługą lokalną i zdalną – na PC z emulatorem ESP8266

Czas	2 jednostki laboratoryjne (zajęcia 4: teoria, przykłady, projekt i początek implementacji; zajęcia 5: dokończenie, testy i oddanie)
Środowisko	Python 3.10+, pakiet startowy: `machine.py`, `network.py`, `telnet_srv.py`, `klient_telnet.py`, `upper_server.py`, `automat_telnet.py`, `test_przekaznik.py`; opcjonalnie `matplotlib`
Platforma docelowa	ESP8266 w trybie punktu dostępowego (TELNET, port 23) + przycisk (D5), LED „cewka" (D6), LED ostrzegawczy (D7), opcjonalnie moduł przekaźnika
Oddajesz	prezentacja na zajęciach 5 + sprawozdanie PDF + pliki `przekaznik.py`, `main.py`

WARUNEK PRZYSTĄPIENIA

Rozumienie mechanizmu działania przekaźnika czasowego jest **warunkiem koniecznym** realizacji zadania. Przed zajęciami 4 przeczytaj część I tej instrukcji oraz dokumentację producenta wybranego przekaźnika (F&F: fif.com.pl → wyszukaj *PCR-515* lub *PCU-520* → karta katalogowa / instrukcja). Prowadzący może sprawdzić znajomość przebiegów czasowych przed rozpoczęciem pracy.

Cel i zagadnienia

Celem zajęć jest zbudowanie **przekaźnika czasowego**, który można obsługiwać **lokalnie** (przycisk) i **zdalnie przez sieć WiFi** (sesja TELNET). Zagadnienia:

- obsługa modułu WiFi ESP8266 w trybie **punktu dostępowego (AP)**, gniazda sieciowe (`socket`), protokół **TELNET**,
- układy RLC – podstawy; przekaźniki czasowe: budowa, zasada działania, typy, zastosowania, przebiegi czasowe,
- obsługa GPIO (wejście z przyciskiem, eliminacja drgań styków), funkcje czasowe,
- **jedna pętla programu** obsługująca jednocześnie przycisk, odmierzanie czasu i polecenia z sieci (nieblokujący odczyt z gniazda).
- automatyczne testy przebiegów czasowych na emulatorze.

Część I – teoria

1. Komunikacja: WiFi i TELNET

1.1. Tryby pracy WiFi w ESP8266

Tryb	Interfejs	Działanie	Adres na ESP8266	W emulatorze
STA (*station*)	`network.STA_IF`	ESP łączy się z istniejącą siecią	nadany przez router (DHCP)	„połączono" (symulacja)
AP (*access point*)	`network.AP_IF`	ESP **tworzy własną sieć**	**192.168.4.1**	127.0.0.1 (ten komputer)

Konfiguracja punktu dostępowego (identyczna na PC i ESP8266):

```
import network
ap = network.WLAN(network.AP_IF)
ap.active(True)
ap.config(essid="SW_Grupa3_Kowalski", password="haslo1234",
          authmode=network.AUTH_WPA_WPA2_PSK)   # hasło: min. 8 znaków!
print(ap.ifconfig())
```

```
[SIM] WLAN AP: aktywny
[SIM] AP: essid='SW_Grupa3_Kowalski', zabezpieczenie: WPA/WPA2-PSK, kanał 1
[SIM] Na ESP8266 adres AP to 192.168.4.1; w symulacji łącz się z 127.0.0.1
('127.0.0.1', '255.0.0.0', '127.0.0.1', '127.0.0.1')
```

Emulator sprawdza konfigurację tak jak urządzenie – np. dla hasła krótszego niż 8 znaków wypisze `[SIM] BŁĄD KONFIGURACJI: hasło WPA musi mieć co najmniej 8 znaków`.

1.2. TELNET i gniazda

TELNET to prosty protokół tekstowy na TCP, domyślnie **port 23**. Na PC porty poniżej 1024 wymagają uprawnień administratora, dlatego w symulacji używamy portu **2323**. Port wybieramy zależnie od platformy:

```
import sys
NA_ESP = sys.platform == "esp8266"
PORT = 23 if NA_ESP else 2323
```

Krok	Funkcja	Uwagi
utworzenie gniazda	`s = socket.socket()`	domyślnie TCP
powiązanie z adresem i portem	`s.bind(socket.getaddrinfo("0.0.0.0", PORT)[0][-1])`	0.0.0.0 = wszystkie interfejsy
nasłuch	`s.listen(1)`	1 klient w kolejce
przyjęcie klienta	`cl, addr = s.accept()`	blokuje do czasu połączenia (z `settimeout(1)` – co 1 s zgłasza `OSError`)
odbiór / wysyłka	`cl.recv(n)`, `cl.send(b"...")`	dane to **bajty** – `str.encode()` / `bytes.decode()`
zamknięcie	`cl.close()`	

Klient TELNET wysyła na końcu linii `\r\n`, a na początku połączenia może wysłać bajty negocjacji opcji (zaczynające się od `0xFF`) – funkcja `czytaj_linie()` w przykładzie je odfiltrowuje.

1.3. Jak uruchomić serwer i klienta na jednym komputerze

Otwórz **dwa** okna terminala w folderze z pakietem:

Okno 1 – serwer	Okno 2 – klient (jedna z opcji)
`python upper_server.py`	`python klient_telnet.py` (dołączony do pakietu)
	`telnet 127.0.0.1 2323` (Linux/macOS; Windows – po włączeniu „Klient Telnet")
	PuTTY: Host `127.0.0.1`, Port `2323`, *Connection type*: **Raw**

Serwer zatrzymasz w oknie 1 klawiszami **Ctrl+C**.

1.4. Nieblokujący odczyt poleceń – moduł `telnet_srv.py`

W przekaźniku program musi **jednocześnie** reagować na przycisk, odmierzać czas i odbierać polecenia z sieci. Zwykłe `cl.recv()` zatrzymuje program do czasu nadejścia danych, dlatego gniazdo klienta przełączamy w tryb nieblokujący (`cl.setblocking(False)`): `recv()` zwraca dane, jeśli są, a w przeciwnym razie zgłasza `OSError` (EAGAIN), który ignorujemy. Gotowe funkcje zebrano w module **`telnet_srv.py`** (pakiet startowy) – możesz z niego korzystać w swoim rozwiązaniu:

Funkcja / metoda	Działanie
`uruchom_ap(ssid, haslo)`	włącza punkt dostępowy WiFi, zwraca adres IP
`gniazdo_serwera()`	tworzy gniazdo serwera na porcie `PORT` (23 na ESP8266, 2323 na PC)
`czekaj_na_klienta(s)`	czeka na połączenie i zwraca obiekt `Polaczenie`
`pol.linia()`	**nie blokuje**: zwraca kompletną linię tekstu od klienta albo `None`
`pol.czekaj_linie()`	czeka na linię (np. przy wpisywaniu nastaw w MENU); `None` po rozłączeniu
`pol.wyslij(tekst)`, `pol.zacheta("> ")`	wysyła linię / tekst bez końca linii
`pol.otwarte`	`False`, gdy klient się rozłączył

Schemat pętli obsługującej wszystko naraz:

```
while pol.otwarte:
    if wcisniecie():                # 1. przycisk lokalny
        ...
    linia = pol.linia()             # 2. polecenie z sieci (nie blokuje!)
    if linia == "stop":
        ...
    if time.ticks_diff(time.ticks_ms(), t0) >= czas_odcinka:   # 3. odmierzanie czasu
        ...
    time.sleep_ms(5)
```

2. Układy RLC – podstawy

Element	Zależność	Znaczenie w układach czasowych
Rezystor **R**	U = I · R	ogranicza prąd (np. diody LED), z kondensatorem tworzy układ RC
Kondensator **C**	i = C · du/dt	gromadzi ładunek; napięcie na nim **nie może zmienić się skokowo**
Cewka **L**	u = L · di/dt	gromadzi energię w polu magnetycznym; prąd w niej **nie może zmienić się skokowo**

Ładowanie kondensatora przez rezystor: $u_C(t) = U \cdot (1 - e^{-t/\tau})$, gdzie **stała czasowa τ = R·C**. Po czasie τ kondensator ma ok. 63 % napięcia zasilania, po 5τ – ponad 99 %. Klasyczne analogowe przekaźniki czasowe wykorzystywały właśnie to zjawisko: gdy napięcie na kondensatorze osiągnie próg U_p, komparator załącza przekaźnik. Czas opóźnienia: $t = R \cdot C \cdot \ln(U / (U - U_p))$. Przykład: R = 100 kΩ, C = 100 µF → τ = 10 s; dla progu 63 % opóźnienie ≈ 10 s. Zmieniając R potencjometrem, regulujemy czas.

Cewka przekaźnika to element RL: prąd narasta ze stałą czasową τ = L/R, a przy nagłym wyłączeniu indukuje się wysokie napięcie (u = L·di/dt), które może uszkodzić tranzystor sterujący. Dlatego równolegle do cewki montuje się **diodę gaszącą** (zaporowo względem zasilania). Gotowe moduły przekaźników mają ją na płytce.

PRZYKŁAD OBLICZENIOWY – DOBÓR REZYSTORA DLA LED

Pin GPIO daje 3,3 V, czerwona dioda LED ma napięcie przewodzenia $U_F \approx 2{,}0$ V, chcemy prądu ok. 6 mA (bezpiecznie dla ESP8266):

$R = (U_{GPIO} - U_F) / I = (3{,}3\text{ V} - 2{,}0\text{ V}) / 0{,}006\text{ A} \approx 217\ \Omega$ → wybieramy wartość z szeregu: **220 Ω**.
Moc: $P = I^2 \cdot R = 0{,}006^2 \cdot 220 \approx 8$ mW – wystarczy rezystor 0,125 W. Dla diody niebieskiej/białej ($U_F \approx 3{,}0$ V) prąd przy 220 Ω wyniesie tylko ok. 1,4 mA – dioda będzie świecić słabo; dobierz mniejszy rezystor (np. 47–68 Ω).

3. Przekaźniki czasowe

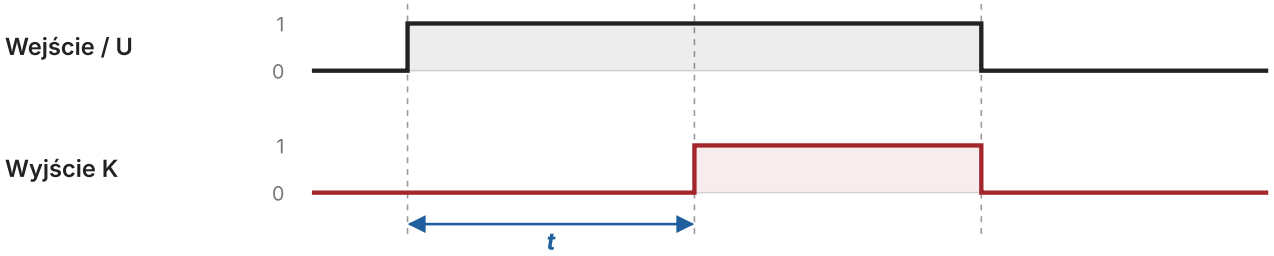
3.1. Budowa i zasada działania

Przekaźnik elektromagnetyczny składa się z **cewki** (elektromagnesu), **kotwicy** ze sprężyną powrotną oraz **styków**: wspólnego (COM), normalnie otwartego (NO) i normalnie zamkniętego (NC). Prąd w cewce przyciąga kotwicę i przełącza styki: COM–NC → COM–NO.

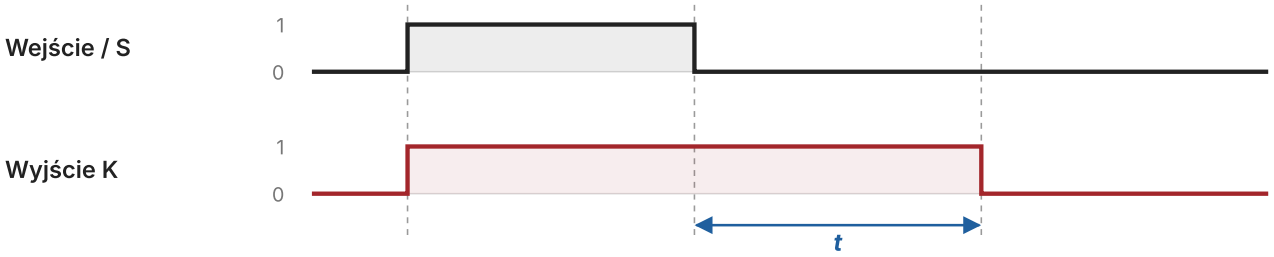
Przekaźnik czasowy to przekaźnik z dodanym **układem czasowym**, który decyduje, *kiedy* i *na jak długo* zasilić cewkę przekaźnika wykonawczego. Typowe elementy: zaciski zasilania (A1–A2), opcjonalne wejście sterujące S, pokrętła nastaw (zakres czasu, czas, funkcja), dioda **U** (zasilanie) i dioda **R** (stan styków), styki wyjściowe (np. 11-12-14 lub 1-5-6). W urządzeniach współczesnych układem czasowym jest mikrokontroler – czyli dokładnie to, co zbudujesz na zajęciach.

3.2. Podstawowe typy i przebiegi czasowe

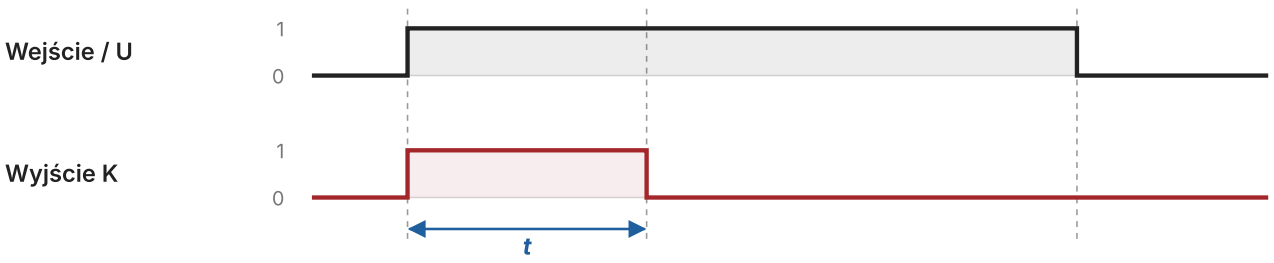
Na przebiegach poziom 1 oznacza obecność sygnału (zasilania/sterowania) lub zasilenie cewki (styki w położeniu „załączonym").



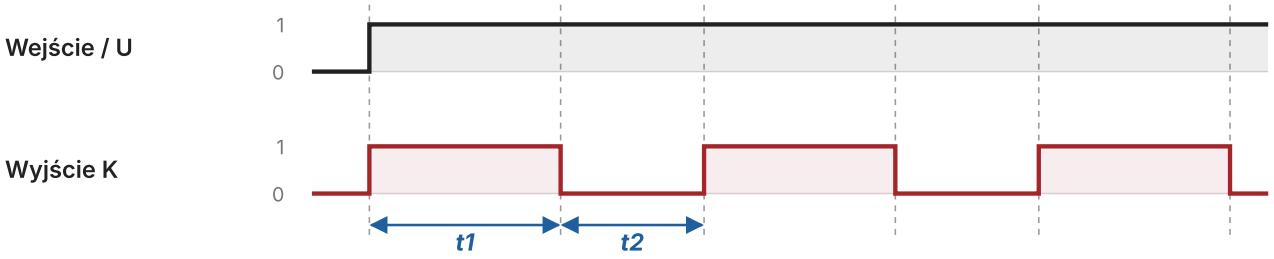
Opóźnione załączenie (ON-delay, TON): wyjście załącza się po czasie t od pojawienia się sygnału.



Opóźnione wyłączenie (OFF-delay, TOF): wyjście wyłącza się po czasie t od zaniku sygnału.



Przekaźnik impulsowy (TP): po pojawieniu się sygnału wyjście jest załączone przez czas t.



Przekaźnik cykliczny asymetryczny: praca t1, przerwa t2, powtarzane do zaniku zasilania.

Inne typy: przekaźnik cykliczny symetryczny (t1 = t2), gwiazda-trójkąt (rozruch silników), przekaźniki **wielofunkcyjne** (funkcję wybiera się przełącznikiem – np. PCU-520 ma cztery).

3.3. Zastosowania

Typ	Przykładowe zastosowanie
opóźnione załączenie	sekwencyjne załączanie odbiorników (ograniczenie prądu rozruchu), opóźnione uruchomienie wentylacji, zabezpieczenie przed zbyt szybkim ponownym startem sprężarki
opóźnione wyłączenie	wentylator łazienkowy działający jeszcze kilka minut po zgaszeniu światła, wybieg pompy
impulsowy	automat schodowy, dzwonek, sygnał syreny o zadanej długości
cykliczny	nawadnianie, mieszadła, sygnalizacja błyskowa, okresowe smarowanie maszyn

4. Obsługa GPIO – wejście z przyciskiem

```
from machine import Pin
btn = Pin(14, Pin.IN, Pin.PULL_UP)   # D5; przycisk zwiera pin do GND
btn.value()                          # 1 = puszczony, 0 = wciśnięty (logika odwrócona)
```

W emulatorze przycisk wciskasz klawiaturą: **Enter** = krótkie wciśnięcie (0,2 s), **l + Enter** = długie (2,5 s), **liczba + Enter** = wciśnięcie na tyle ms (np. `1500`). W testach zamiast klawiatury używasz scenariusza: `machine.sim_button(14, [(1000, 200), (5000, 200)])`. Emulator nie symuluje drgań styków – ale Twój kod i tak musi je eliminować, bo na urządzeniu wystąpią.

Drgania styków: przycisk mechaniczny przy wciśnięciu przez 1–20 ms wielokrotnie zwiera i rozwiera styki. Program odczytujący pin bardzo często „zobaczy" kilka wciśnięć. Rozwiązanie programowe – po wykryciu zmiany odczekaj ok. 30 ms i odczytaj ponownie; reaguj tylko na **zbocze** (zmianę 1 → 0), a nie na sam stan.

```python
import time
_poprzedni = 1
def wcisniecie():
    """Zwraca True dokładnie raz na każde wciśnięcie przycisku (z eliminacją drgań)."""
    global _poprzedni
    stan = btn.value()
    if stan != _poprzedni:
        time.sleep_ms(30)              # przeczekaj drgania
        stan = btn.value()
        if stan != _poprzedni:
            _poprzedni = stan
            return stan == 0           # zbocze opadające = wciśnięcie
    return False
```

Alternatywa: przerwanie `btn.irq(trigger=Pin.IRQ_FALLING, handler=funkcja)` – funkcja obsługi powinna być bardzo krótka (np. tylko ustawiać flagę).

5. Odmierzanie czasu bez blokowania i maszyna stanów

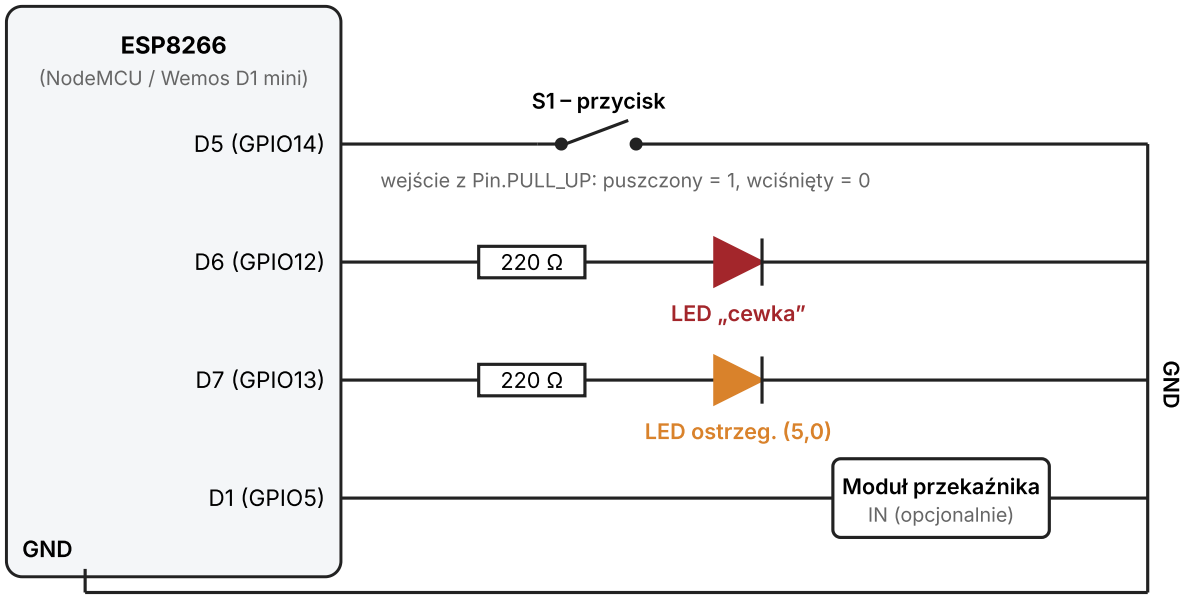
Przekaźnik musi **w każdej chwili** reagować na przycisk (przerwanie pracy), więc **nie używaj** `time.sleep(t1)` do odmierzania czasów t1, t2. Zamiast tego w pętli sprawdzaj, ile czasu minęło od początku odcinka:

```python
t0 = time.ticks_ms()
while time.ticks_diff(time.ticks_ms(), t0) < t1_ms:
    if wcisniecie():
        ...                # przerwanie pracy
    time.sleep_ms(5)       # krótka pauza – odciąża procesor
```

Program przekaźnika najwygodniej zaprojektować jako **maszynę stanów** (np. `MENU`, `MANUAL`, `AUTO_CZEKA_NA_START`, `AUTO_ODCINEK_T1`, `AUTO_ODCINEK_T2`). Narysuj diagram stanów przed pisaniem kodu – zamieścisz go w sprawozdaniu.

6. Przypisanie pinów i schemat docelowego układu

Program piszesz dla docelowego układu poniżej – na emulatorze używasz tych samych numerów GPIO. Schemat przyda się przy przeniesieniu na płytkę i w sprawozdaniu (dobór rezystora – p. 2).



Element	Pin płytki	GPIO	W emulatorze
przycisk S1 (do GND, `Pin.PULL_UP`)	D5	14	klawiatura (Enter) lub `sim_button(14, …)`
LED „cewka" = stan przekaźnika K	D6	12	wpisy w logu; `sim_intervals(12)`
LED ostrzegawczy (wariant 5,0)	D7	13	wpisy w logu; `sim_intervals(13)`
moduł przekaźnika (opcjonalnie)	D1	5	sterowany równolegle z D6

Nazwy w logach ustawisz w pliku testu lub uruchomieniowym: `machine.sim_config(names={12: ("K", False), 13: ("LED ostrz.", False)})`.

Część II – przykłady rozwiązań (inne zadania)

PRZYKŁAD 1 – SERWER TELNET ZAMIENIAJĄCY TEKST NA WIELKIE LITERY (PLIK UPPER_SERVER.PY)

Treść: Urządzenie tworzy sieć WiFi i uruchamia serwer TELNET. Każda linia od klienta wraca wielkimi literami wraz z liczbą znaków. `help` wyświetla pomoc, `quit` kończy sesję (serwer czeka na kolejnego klienta). Logika odpowiedzi jest w osobnej funkcji `obsluz_linie()`, którą można testować bez sieci.

```
import sys
import network
import socket
import time

NA_ESP = sys.platform == "esp8266"
SSID, HASLO = "SW_Przyklad", "haslo1234"
PORT = 23 if NA_ESP else 2323          # na PC porty < 1024 wymagają uprawnień
administratora

def uruchom_ap(ssid, haslo):
    network.WLAN(network.STA_IF).active(False)        # wyłącz tryb STA
    ap = network.WLAN(network.AP_IF)
    ap.active(True)
    ap.config(essid=ssid, password=haslo, authmode=network.AUTH_WPA_WPA2_PSK)
    while not ap.active():
        time.sleep(0.1)
    print("AP '{}' aktywny, IP: {}".format(ssid, ap.ifconfig()[0]))

def czytaj_linie(cl):
    """Czyta znaki do końca linii. Zwraca tekst lub None, gdy klient się rozłączył."""
    buf = b""
    while True:
        b = cl.recv(1)
        if not b:                      # połączenie zamknięte
            return None
        if b == b"\xff":               # IAC – negocjacja TELNET: pomiń 2 kolejne bajty
            cl.recv(2)
            continue
        if b == b"\n":
            return buf.decode().strip()
        if b not in (b"\r", b"\x00"):
            buf += b

def wyslij(cl, tekst):
    cl.send((tekst + "\r\n").encode())

def obsluz_linie(linia):
    """Logika serwera – czysta funkcja: tekst -> odpowiedź (łatwa do testowania bez
sieci)."""
    if linia.lower() == "help":
        return "Dowolny tekst -> wielkie litery; quit - koniec sesji"
    return "{} ({} znakow)".format(linia.upper(), len(linia))

def obsluz_klienta(cl):
    wyslij(cl, "Serwer UPPER. Wpisz 'help'.")
    while True:
        cl.send(b"> ")
        linia = czytaj_linie(cl)
        if linia is None or linia.lower() == "quit":
            break
        if linia:
            wyslij(cl, obsluz_linie(linia))
    wyslij(cl, "Do widzenia!")

def serwer():
```

```
    s = socket.socket()
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(socket.getaddrinfo("0.0.0.0", PORT)[0][-1])
    s.listen(1)
    s.settimeout(1)                 # accept() co 1 s oddaje sterowanie – Ctrl+C działa
także na Windows
    print("Serwer TELNET nasluchuje na porcie", PORT)
    while True:
        try:
            cl, addr = s.accept()
        except OSError:             # upłynął czas oczekiwania – czekamy dalej
            continue
        cl.settimeout(None)
        print("Polaczenie z", addr)
        try:
            obsluz_klienta(cl)
        except OSError as e:        # np. zerwane połączenie
            print("Blad polaczenia:", e)
        cl.close()
        print("Klient rozlaczony")

uruchom_ap(SSID, HASLO)
serwer()
```

Okno 1 – serwer:

```
[SIM] WLAN STA: wyłączony
[SIM] WLAN AP: aktywny
[SIM] AP: essid='SW_Przyklad', zabezpieczenie: WPA/WPA2-PSK, kanał 1
[SIM] Na ESP8266 adres AP to 192.168.4.1; w symulacji łącz się z 127.0.0.1
AP 'SW_Przyklad' aktywny, IP: 127.0.0.1
Serwer TELNET nasluchuje na porcie 2323
Polaczenie z ('127.0.0.1', 48646)
Klient rozlaczony
```

Okno 2 – klient (`python klient_telnet.py`):

```
Połączono z 127.0.0.1:2323
Serwer UPPER. Wpisz 'help'.
> ala ma kota
ALA MA KOTA (11 znakow)
> help
Dowolny tekst -> wielkie litery; quit - koniec sesji
> quit
Do widzenia!
[Serwer zamknął połączenie]
```

PRZYKŁAD 2 – AUTOMAT SCHODOWY OBSŁUGIWANY PRZYCISKIEM I PRZEZ TELNET (PLIK AUTOMAT_TELNET.PY)

Treść: Każde wciśnięcie przycisku **lub** polecenie `on` wysłane przez TELNET zapala lampę (LED) na czas t; ponowne wciśnięcie odlicza czas od nowa. Przez TELNET można też zmienić czas (`t 5`), sprawdzić stan (`status`) i zakończyć sesję (`quit`). Każde zdarzenie trafia do klienta i na konsolę. Po rozłączeniu klienta lampa gaśnie (stan bezpieczny), a serwer czeka na kolejne połączenie. Uruchom `python automat_telnet.py` w jednym oknie i `python klient_telnet.py` w drugim. Przycisk wciskasz klawiszem Enter **w oknie serwera**.

```
# automat_telnet.py — automat schodowy: przycisk + TELNET
from machine import Pin
import time
import telnet_srv as tn

SSID, HASLO = "SW_Automat", "haslo1234"
PIN_PRZYCISK, PIN_LAMPA = 14, 12          # D5, D6
T_MIN, T_MAX = 1, 60                      # zakres nastawy [s]
POMOC = "Polecenia: t <s> - czas swiecenia (1-60), on - zdalne wlaczenie, status, help,
quit"

btn = Pin(PIN_PRZYCISK, Pin.IN, Pin.PULL_UP)
lampa = Pin(PIN_LAMPA, Pin.OUT, value=0)

_poprzedni = 1
def wcisniecie():
    """Zwraca True dokładnie raz na każde wciśnięcie (zbocze 1->0, eliminacja drgań)."""
    global _poprzedni
    stan = btn.value()
    if stan != _poprzedni:
        time.sleep_ms(30)
        stan = btn.value()
        if stan != _poprzedni:
            _poprzedni = stan
            return stan == 0
    return False

def sprawdz_czas(tekst, tmin, tmax):
    """Czysta funkcja walidacji: zwraca (czas, None) albo (None, komunikat_bledu)."""
    try:
        t = float(tekst.strip().replace(",", "."))
    except ValueError:
        return None, "Blad: wpisz liczbe, np. 5 lub 2.5"
    if not tmin <= t <= tmax:
        return None, "Blad: dozwolone wartosci {}-{} s".format(tmin, tmax)
    return t, None

def sesja(pol, t_ms):
    """Obsługa jednego klienta: jedna pętla obsługuje przycisk, czas i polecenia z
sieci."""
    pol.wyslij("=== AUTOMAT SCHODOWY (sterowanie zdalne) ===")
    pol.wyslij(POMOC)
    start = time.ticks_ms()
    zapalona_od = None                    # None = lampa zgaszona

    def zdarzenie(tekst):                 # log na konsoli i u klienta
        msg = "[{:7.2f} s] {}".format(time.ticks_diff(time.ticks_ms(), start) / 1000,
tekst)
        print(msg)
        pol.wyslij(msg)

    while pol.otwarte:
        wlacz = wcisniecie()
        linia = pol.linia()               # NIE blokuje
        if linia is not None:
```

```
            cmd = linia.lower().split()
            if not cmd:
                pass
            elif cmd[0] == "quit":
                break
            elif cmd[0] == "help":
                pol.wyslij(POMOC)
            elif cmd[0] == "on":
                wlacz = True
            elif cmd[0] == "status":
                if zapalona_od is None:
                    pol.wyslij("Lampa zgaszona, t = {} s".format(t_ms / 1000))
                else:
                    zostalo = t_ms - time.ticks_diff(time.ticks_ms(), zapalona_od)
                    pol.wyslij("Lampa swieci, do zgasniecia {:.1f} s".format(zostalo /
1000))
            elif cmd[0] == "t" and len(cmd) == 2:
                t, blad = sprawdz_czas(cmd[1], T_MIN, T_MAX)
                if blad:
                    pol.wyslij(blad)
                else:
                    t_ms = int(t * 1000)
                    pol.wyslij("Nowy czas swiecenia: {} s".format(t))
            else:
                pol.wyslij("Nieznane polecenie. " + POMOC)
        if wlacz:
            zapalona_od = time.ticks_ms()
            lampa.value(1)
            zdarzenie("lampa ON na {} s".format(t_ms / 1000))
        if zapalona_od is not None and time.ticks_diff(time.ticks_ms(), zapalona_od) >=
t_ms:
            lampa.value(0)
            zapalona_od = None
            zdarzenie("lampa OFF")
        time.sleep_ms(5)
    lampa.value(0)                          # stan bezpieczny po zakończeniu sesji
    pol.wyslij("Do widzenia!")
    return t_ms

def main():
    tn.uruchom_ap(SSID, HASLO)
    s = tn.gniazdo_serwera()
    t_ms = 10000
    while True:
        pol = tn.czekaj_na_klienta(s)
        t_ms = sesja(pol, t_ms)
        pol.zamknij()
        print("Klient rozlaczony")

main()
```

Sesja klienta TELNET (wpisane polecenia: `help`, `t 0.5`, `t 2`, `on`, `status`, `xyz`, `quit`):

```
=== AUTOMAT SCHODOWY (sterowanie zdalne) ===
Polecenia: t <s> - czas swiecenia (1-60), on - zdalne wlaczenie, status, help, quit
help
Polecenia: t <s> - czas swiecenia (1-60), on - zdalne wlaczenie, status, help, quit
t 0.5
Blad: dozwolone wartosci 1-60 s
t 2
Nowy czas swiecenia: 2.0 s
on
[   0.02 s] lampa ON na 2.0 s
status
Lampa swieci, do zgasniecia 1.0 s
[   2.02 s] lampa OFF
xyz
Nieznane polecenie. Polecenia: t <s> - czas swiecenia (1-60), on - zdalne wlaczenie,
```

```
status, help, quit
quit
Do widzenia!
```

Co warto przenieść do swojego rozwiązania: jedną pętlę obsługującą przycisk, sieć i czas; walidację w czystej funkcji `sprawdz_czas()`; wysyłanie każdego zdarzenia z czasem do klienta i na konsolę; stan bezpieczny po rozłączeniu.

W pakiecie znajdziesz też **automat.py** z testem **test_automat.py** – ten sam automat bez sieci, z przykładem testu przebiegu czasowego w czasie wirtualnym (`sim_button` , `sim_intervals` , `sim_ascii`). Na tej samej zasadzie działa `test_przekaznik.py` .

Część III – zadanie: przekaźnik czasowy sterowany przez WiFi

Wybierz **jeden wariant** zadania. Ocena zależy od wybranego wariantu oraz jakości realizacji (poprawność przebiegów czasowych, obsługa błędów, działanie sterowania zdalnego, czytelność kodu, sprawozdanie).

Wariant	Przekaźnik	Tryby pracy	Ocena maks.
A	PCR-515 DUO – opóźnione załączenie	AUTO, MANUAL	3,0
B	PCU-520 – asymetryczny, funkcje A–D	AUTO, MANUAL_BI, MANUAL_MONO	4,0
C	jak wariant B + LED ostrzegawczy	jak wariant B	5,0

Wymagania wspólne dla wszystkich wariantów

Sieć

1. Po uruchomieniu program konfiguruje **punkt dostępowy** o unikalnej nazwie (np. `SW_<grupa>_<nazwisko>`), zabezpieczony hasłem (min. 8 znaków), i wypisuje adres IP na konsoli.
2. Działa serwer TELNET na porcie 23 (na PC w emulatorze 2323 – wybór przez `sys.platform`). Obsługiwany jest jeden klient naraz; po jego rozłączeniu serwer czeka na kolejnego.
3. **MENU, wybór trybu i nastawy czasów odbywają się w sesji TELNET.** Konsola urządzenia (REPL) służy tylko do logów.
4. Polecenia dostępne w każdej chwili: `help` (lista poleceń), `status` (tryb, funkcja, stan K, czas pozostały do końca odcinka), `quit` (koniec sesji).
5. Każda zmiana stanu cewki K jest wysyłana do klienta i wypisywana na konsoli z czasem od startu cyklu.
6. Rozłączenie klienta lub `quit` → K = 0 i LED ostrzegawczy zgaszony (stan bezpieczny).

Przekaźnik

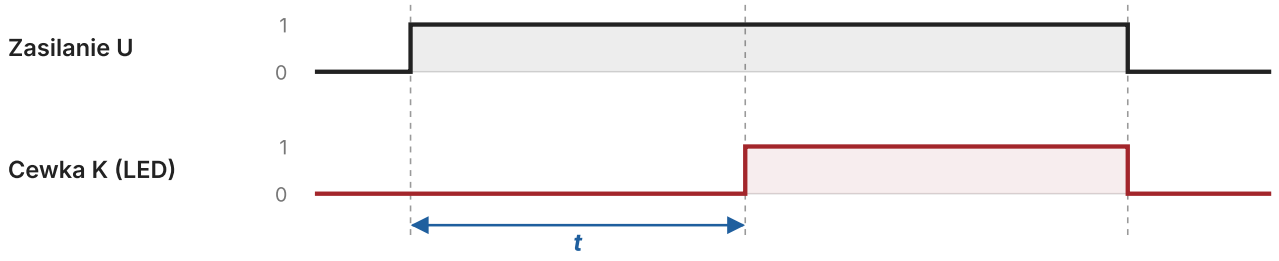
7. Numery pinów i zakresy czasów zdefiniowane jako stałe na początku programu.
8. Czasy odmierzane **nieblokująco** (`ticks_ms` / `ticks_diff`); w trakcie cyklu program reaguje jednocześnie na przycisk i polecenia z sieci (`pol.linia()`).
9. Zakres nastaw czasów: **0,1–60,0 s** z rozdzielczością 0,1 s (akceptuj także przecinek dziesiętny). Wartości błędne lub spoza zakresu są odrzucane z komunikatem, jakie wartości są dozwolone.

10. Start cyklu AUTO: wciśnięcie przycisku **lub** polecenie `start`. Przerwanie pracy: przycisk **lub** polecenie `stop` → powrót do MENU.
11. Przycisk z eliminacją drgań styków; reakcja na zbocze (jedno wciśnięcie = jedno zdarzenie).
12. **Interfejs modułu** `przekaznik.py` (sprawdzany testem `test_przekaznik.py`):
    - stałe `PIN_PRZYCISK = 14`, `PIN_K = 12`, `PIN_OSTRZ = 13`,
    - `odcinki(funkcja, t1_ms, t2_ms)` – czysta funkcja zwracająca jeden cykl przebiegu jako listę `(stan_K, czas_ms)`; czas `None` = „do przerwania". Np. `odcinki("PCR", 3000, 0) == [(0, 3000), (1, None)]`, `odcinki("A", 2000, 1000) == [(1, 2000), (0, 1000)]`, `odcinki("B", 2000, 1000) == [(0, 2000), (1, 1000)]` (C i D – jak A i B, powtarzane),
    - `praca_auto(funkcja, t1_ms, t2_ms, pol=None)` – realizuje tryb AUTO od chwili wywołania (start obsługuje kod wywołujący); kończy się, zwracając `"przerwano"` po wciśnięciu przycisku (lub poleceniu `stop`, gdy podano połączenie `pol`) albo `"koniec"` po zakończeniu funkcji jednorazowej (A, B); przy wyjściu K = 0. Gdy `pol` jest podane, zdarzenia są wysyłane także do klienta,
    - `main()` – punkt dostępowy, serwer i obsługa sesji; `main.py` zawiera tylko `import przekaznik` i `przekaznik.main()`.
13. **Warunek zaliczenia:** `python test_przekaznik.py` (z ustawionym `WARIANT`) kończy się komunikatem „wszystkie testy zaliczone", a sterowanie przez TELNET działa podczas prezentacji.

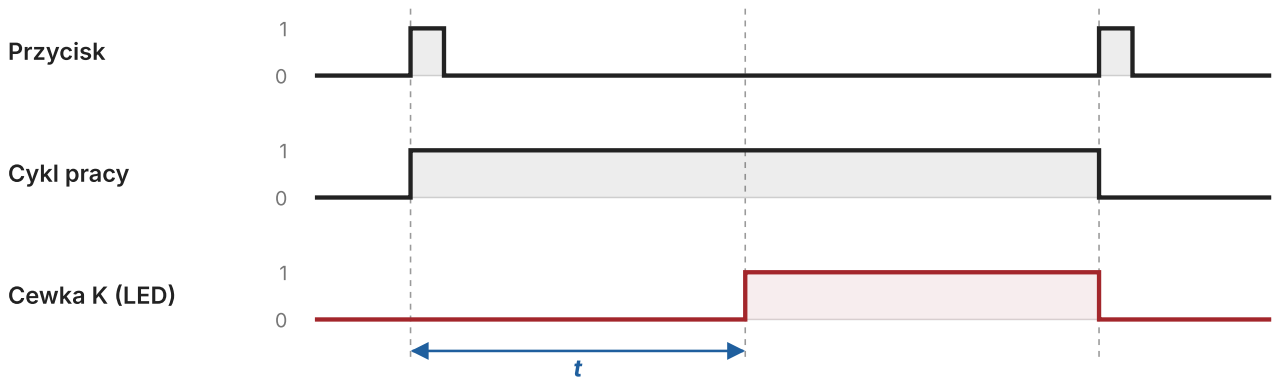
WARIANT A – OCENA 3,0

Zaimplementuj dla ESP8266 – uruchamiając i testując na emulatorze – przekaźnik czasowy realizujący funkcje tożsame z przekaźnikiem **PCR-515 DUO z opóźnionym załączeniem** (F&F), zgodny z przebiegami czasowymi producenta, z dwoma trybami pracy: **AUTO** i **MANUAL**, obsługiwany przez TELNET.

1. Po połączeniu klient widzi **MENU** zawierające nazwę przekaźnika, sposób działania (wg producenta) oraz przebieg czasowy (w formie tekstowej) z podpisanym parametrem t.
2. MENU pozwala wybrać tryb: AUTO / MANUAL. AUTO realizuje przebieg czasowy z dokumentacji producenta.
3. MANUAL – sterowanie **bistabilne**: każde wciśnięcie przycisku **lub** polecenie `t` (toggle) zmienia stan przekaźnika; polecenie `menu` wraca do MENU.
4. AUTO – program prosi o wartość t (walidacja wg wymagań wspólnych), a następnie informuje, że cykl rozpocznie się po wciśnięciu przycisku lub poleceniu `start` (= „podanie zasilania").
5. Stan cewki K sygnalizuje LED „cewka" (GPIO12): świeci wtedy i tylko wtedy, gdy cewka jest zasilana; w sprawozdaniu dobierz rezystor tej diody dla układu docelowego.
6. Wciśnięcie przycisku lub polecenie `stop` („zanik zasilania") przerywa pracę i wyświetla MENU.



PCR-515: po podaniu U odliczany jest czas t, potem K załącza się i pozostaje załączona aż do zaniku U.



Symulacja: 1. wciśnięcie = „podanie zasilania U", 2. wciśnięcie = „zanik U" → powrót do MENU.

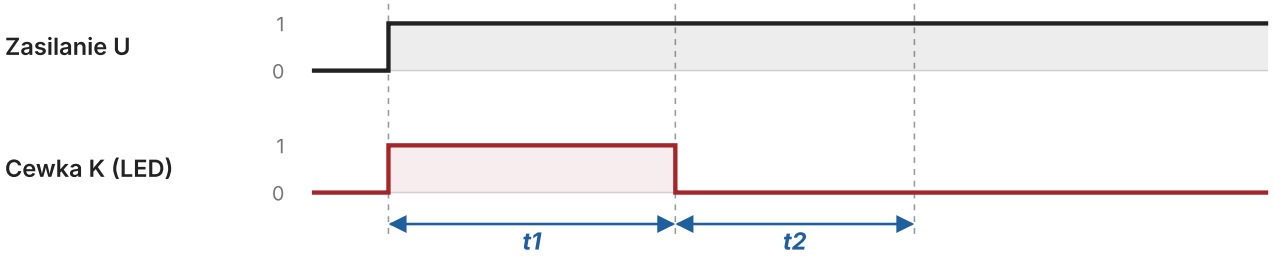
WARIANT B – OCENA 4,0

Zaimplementuj dla ESP8266 – uruchamiając i testując na emulatorze – przekaźnik czasowy realizujący funkcje tożsame z **asymetrycznym przekaźnikiem czasowym PCU-520** (F&F), zgodny z przebiegami czasowymi producenta, z trzema trybami pracy: **AUTO / MANUAL_BI / MANUAL_MONO**, obsługiwany przez TELNET.

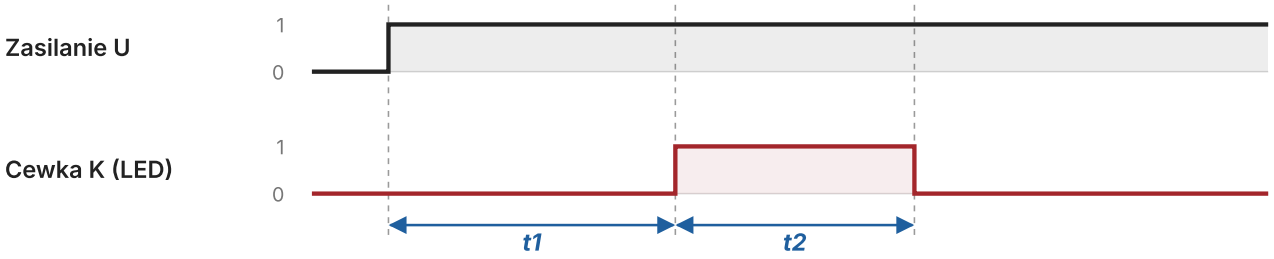
1. Po połączeniu klient widzi MENU zawierające nazwę przekaźnika, sposób działania (wg producenta) oraz przebiegi czasowe z podpisanymi parametrami (t1, t2) – **dla każdej z 4 funkcji A–D**.
2. MENU pozwala wybrać jeden z trzech trybów. MANUAL_BI – sterowanie bistabilne (wciśnięcie przycisku lub polecenie `t` zmienia stan). MANUAL_MONO – sterowanie monostabilne (wciśnięcie przycisku lub polecenie `p` zasila cewkę przez **1 s**).
3. Tryby MANUAL działają aż do resetu układu (na ESP8266: RST; w emulatorze: Ctrl+C) lub zakończenia sesji (`quit`).
4. AUTO – program pozwala wybrać funkcję (**A–D**), a następnie zadać t1 i t2 (walidacja wg wymagań wspólnych; również błędny wybór funkcji). Następnie informuje, że cykl rozpocznie się po wciśnięciu przycisku lub poleceniu `start` .
5. Stan cewki K sygnalizuje LED „cewka" (GPIO12).
6. Przycisk lub `stop` w trakcie pracy przerywa cykl i wyświetla MENU. Po zakończeniu funkcji jednorazowych (A, B) klient dostaje komunikat o końcu cyklu i wraca do MENU.

Funkcje przekaźnika PCU-520 (wg instrukcji producenta; „U" = podanie zasilania, w symulacji – wciśnięcie przycisku startu):

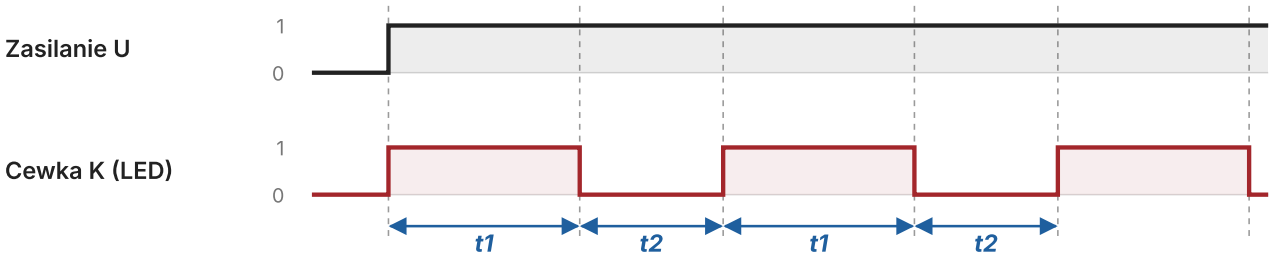
Funkcja	Opis działania
A – opóźnione wyłączenie	po podaniu U styki przełączają się (K = 1) na czas **t1**, następnie wracają (K = 0) na czas **t2**; ponowna realizacja wymaga ponownego podania zasilania
B – opóźnione załączenie	po podaniu U styki pozostają w położeniu spoczynkowym (K = 0) przez **t1**, następnie przełączają się (K = 1) na czas **t2** i wracają do położenia spoczynkowego
C – opóźnione wyłączenie cyklicznie	funkcja A powtarzana cyklicznie: praca t1 / przerwa t2 / praca t1 ...
D – opóźnione załączenie cyklicznie	funkcja B powtarzana cyklicznie: przerwa t1 / praca t2 / przerwa t1 ...



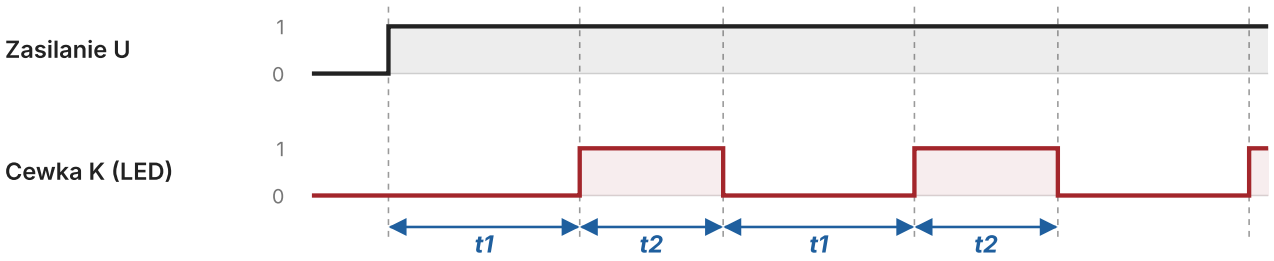
A – opóźnione wyłączenie: K załączona przez t1, potem wyłączona przez t2; koniec cyklu.



B – opóźnione załączenie: przerwa t1, K załączona przez t2, potem wyłączona; koniec cyklu.



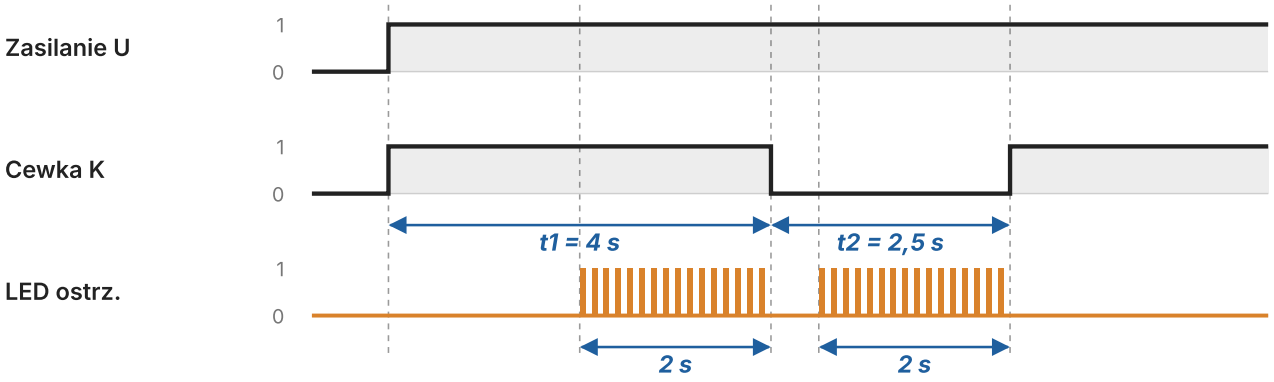
C – opóźnione wyłączenie cyklicznie: praca t1 / przerwa t2, powtarzane aż do przerwania.



D – opóźnione załączenie cyklicznie: przerwa t1 / praca t2, powtarzane aż do przerwania.

WARIANT C – OCENA 5,0

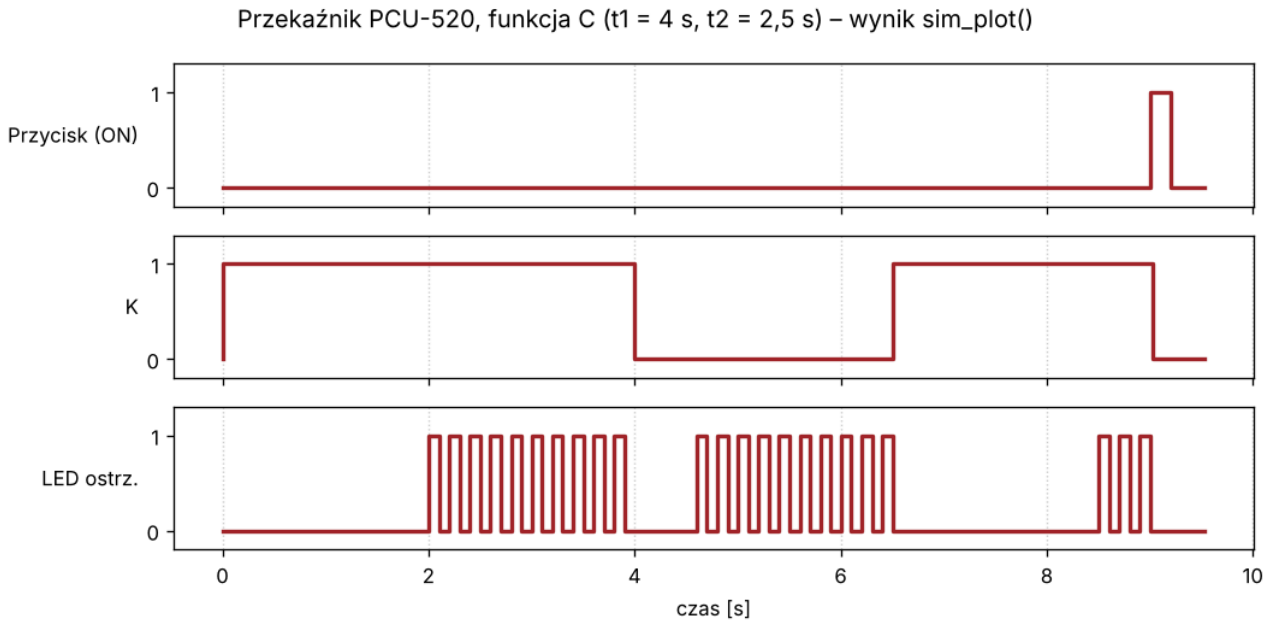
1. Zrealizuj wariant B (wszystkie wymagania).
2. Zastosuj dodatkowy wskaźnik świetlny – LED ostrzegawczy (D7). W trybie AUTO dioda emituje **szybko pulsujące światło przez 2 s przed każdą zmianą stanu przekaźnika** (np. 5 Hz: 100 ms świeci / 100 ms nie świeci). Jeśli odcinek czasu jest krótszy niż 2 s, dioda pulsuje przez cały ten odcinek.
3. Na 2 s przed zmianą stanu K klient TELNET otrzymuje komunikat ostrzegawczy (np. `Uwaga: zmiana stanu K za 2 s`).



Wariant 5,0: LED ostrzegawczy pulsuje szybko (np. 5 Hz) przez 2 s przed każdą zmianą stanu K.

WSKAZÓWKI DO REALIZACJI

- **Zacznij od kartki:** narysuj przebiegi czasowe swojego wariantu i diagram stanów programu (stany: brak klienta, MENU, wprowadzanie nastaw, oczekiwanie na start, odcinek t1, odcinek t2, MANUAL...).
- **Kolejność pracy:** (1) uruchom przykład 2 bez zmian, (2) MENU i wprowadzanie nastaw przez TELNET (`pol.czekaj_linie()`), (3) MANUAL, (4) AUTO z jedną funkcją – najpierw bez sieci, potem z poleceniami `start` / `stop` , (5) pozostałe funkcje, (6) LED ostrzegawczy.
- Funkcje A–D da się opisać wspólnie: każda to ciąg odcinków *(stan K, czas)*, np. A = `[(1, t1), (0, t2)]` , B = `[(0, t1), (1, t2)]` ; C i D to te same listy powtarzane w pętli. Jedna funkcja „odtwarzająca" listę odcinków obsłuży wszystkie warianty.
- MENU w formie tekstowej wyślij do klienta kilkoma wywołaniami `pol.wyslij()` (jedna linia = jedno wywołanie) – przebiegi czasowe narysuj znakami, np. `K : ____|^^^^^^^^|____` oraz `|<- t ->|` .
- W wariancie C pulsowanie licz także nieblokująco: `(ticks_diff(teraz, t0) // 100) % 2` daje naprzemiennie 0 i 1 co 100 ms.
- Sprawdź dokładność: porównaj czasy z logu z wartościami zadanymi (różnica powinna wynosić kilka–kilkanaście ms).
- Uruchamiaj `test_przekaznik.py` po każdym etapie – wskaże, który przebieg jest błędny i jak wygląda zmierzony.
- Do sprawozdania wygeneruj przebieg: w osobnym skrypcie ustaw `sim_config(fast=True)` , scenariusz `sim_button(...)` , wywołaj `praca_auto(...)` i `machine.sim_plot("przebieg_C.png", [14, 12, 13])` (rysunek poniżej).



PRZENIESIENIE NA ESP8266

Wgraj `przekaznik.py`, `telnet_srv.py` i `main.py` (bez `machine.py`, `network.py` i testów), połącz układ wg schematu z p. 6 i uruchom. Laptop połącz z siecią utworzoną przez ESP (zniknie wtedy dostęp do Internetu) i połącz się: `telnet 192.168.4.1` lub `python klient_telnet.py 192.168.4.1 23`. Dzięki `sys.platform` serwer sam użyje portu 23. Na urządzeniu dojdą drgania styków – jeśli przycisk „podwójnie" reaguje, wydłuż czas eliminacji drgań.

Sprawozdanie

Wg wzoru z „Informacji ogólnych", a dodatkowo:

- wybrany wariant i opis przekaźnika (zasada działania, przebiegi czasowe wg producenta – z podaniem źródła),
- parametry sieci (SSID, zabezpieczenie, adres IP na PC i na ESP8266, port), użyty klient TELNET, lista poleceń sesji,
- schemat docelowego układu z oznaczeniem pinów oraz **obliczenie rezystora** dla diod LED,
- diagram stanów programu i opis najważniejszych funkcji,
- zrzuty ekranu sesji TELNET (MENU, nastawy, obsługa błędów, przebieg każdego trybu i każdej funkcji A–D) oraz logów na konsoli urządzenia,
- wynik `test_przekaznik.py` oraz przebiegi czasowe z emulatora (`sim_ascii` lub `sim_plot`) porównane z przebiegami producenta,
- opis przeniesienia na ESP8266 i spodziewanych różnic,
- wnioski (dokładność odmierzania czasu, obsługa sieci i przycisku w jednej pętli, ograniczenia rozwiązania); załącznik: kod programu i testów.

Lista kontrolna przed oddaniem

☐ Punkt dostępowy ma unikalną nazwę i hasło (min. 8 znaków); klient łączy się z serwerem.
☐ MENU w sesji TELNET zawiera nazwę, opis działania i przebieg(i) czasowe z podpisanymi parametrami.
☐ Wszystkie tryby działają; w AUTO przebieg jest zgodny z dokumentacją producenta.
☐ Błędne nastawy i nieznane polecenia nie przerywają pracy – klient dostaje komunikat.
☐ Cykl można uruchomić i przerwać zarówno przyciskiem, jak i poleceniem (`start` / `stop`) w **dowolnym** momencie.
☐ LED „cewka" świeci zawsze wtedy, gdy K = 1 (i tylko wtedy); po rozłączeniu klienta K = 0.
☐ (Wariant C) LED ostrzegawczy pulsuje 2 s przed każdą zmianą stanu K.
☐ `test_przekaznik.py` – wszystkie testy zaliczone; `main.py` tylko uruchamia moduł.

Załącznik A – moduł telnet_srv.py

Plik znajduje się w pakiecie startowym. Działa bez zmian na PC (z emulatorem) i na ESP8266 – wgraj go na płytkę razem ze swoim programem.

```
# telnet_srv.py – moduł pomocniczy (Laboratorium 4): punkt dostępowy WiFi + serwer TELNET
# z NIEBLOKUJĄCYM odczytem linii. Działa bez zmian na ESP8266 (MicroPython) i na PC (z
emulatorem).
# Na ESP8266 wgraj ten plik razem ze swoim programem.
import sys
import machine          # na PC: emulator dodaje do modułu time funkcje sleep_ms/ticks_ms
import network
import socket
import time

NA_ESP = sys.platform == "esp8266"
PORT = 23 if NA_ESP else 2323            # na PC porty < 1024 wymagają uprawnień administratora
_BRAK_DANYCH = (11, 35, 115, 10035)      # EAGAIN / EWOULDBLOCK na różnych systemach


def uruchom_ap(ssid, haslo):
    """Włącza punkt dostępowy WiFi zabezpieczony hasłem (min. 8 znaków). Zwraca adres IP."""
    network.WLAN(network.STA_IF).active(False)
    ap = network.WLAN(network.AP_IF)
    ap.active(True)
    ap.config(essid=ssid, password=haslo, authmode=network.AUTH_WPA_WPA2_PSK)
    while not ap.active():
        time.sleep_ms(100)
    ip = ap.ifconfig()[0]
    print("AP '{}' aktywny, IP: {}".format(ssid, ip))
    return ip


def gniazdo_serwera(port=PORT):
    """Tworzy gniazdo nasłuchujące serwera TCP."""
    s = socket.socket()
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(socket.getaddrinfo("0.0.0.0", port)[0][-1])
    s.listen(1)
    s.settimeout(1)                      # accept() co 1 s oddaje sterowanie (Ctrl+C działa)
    print("Serwer TELNET nasluchuje na porcie", port)
    return s


def czekaj_na_klienta(s):
    """Czeka na połączenie klienta i zwraca obiekt Polaczenie."""
    while True:
        try:
            cl, addr = s.accept()
        except OSError:
            continue
        print("Polaczenie z", addr)
        return Polaczenie(cl)


class Polaczenie:
    """Połączenie z klientem TELNET. Metoda linia() NIE blokuje programu – można ją wywoływać
    w tej samej pętli, w której obsługujesz przycisk i odmierzasz czas."""

    def __init__(self, cl):
        self.cl = cl
        self.buf = b""
        self.otwarte = True
        cl.setblocking(False)
```

```
    def wyslij(self, tekst=""):
        """Wysyła linię tekstu do klienta."""
        self._send((tekst + "\r\n").encode())

    def zacheta(self, tekst="> "):
        """Wysyła tekst bez znaku końca linii (np. znak zachęty)."""
        self._send(tekst.encode())

    def _send(self, dane):
        if not self.otwarte:
            return
        try:
            self.cl.send(dane)
        except OSError:
            self.otwarte = False

    def linia(self):
        """Zwraca kompletną linię tekstu od klienta albo None, jeśli jeszcze jej nie ma.
        Gdy klient się rozłączy, ustawia self.otwarte = False."""
        if self.otwarte:
            try:
                dane = self.cl.recv(64)
                if dane:
                    self.buf += dane
                else:
                    self.otwarte = False             # klient zamknął połączenie
            except OSError as e:
                if not (e.args and e.args[0] in _BRAK_DANYCH):
                    self.otwarte = False             # połączenie zerwane
        while True:                                  # usuń negocjację TELNET (IAC = 0xFF + 2
bajty)
            i = self.buf.find(b"\xff")
            if i < 0:
                break
            self.buf = self.buf[:i] + self.buf[i + 3:]
        i = self.buf.find(b"\n")
        if i < 0:
            return None
        surowa, self.buf = self.buf[:i], self.buf[i + 1:]
        surowa = surowa.replace(b"\r", b"").replace(b"\x00", b"")
        try:
            return surowa.decode().strip()
        except UnicodeError:
            return ""

    def czekaj_linie(self):
        """Czeka (blokująco, z pauzami 10 ms) na linię od klienta; None po rozłączeniu."""
        while self.otwarte:
            l = self.linia()
            if l is not None:
                return l
            time.sleep_ms(10)
        return None

    def zamknij(self):
        try:
            self.cl.close()
        except OSError:
            pass
        self.otwarte = False
```